

ZipCode3.0

Generated by Doxygen 1.13.2

1 ZipCode3.0	1
1.1 ZIP Code Blocked Sequence Set Processor	1
1.2 Description	1
1.3 Features	1
1.4 Project Structure	1
1.5 Build Instructions	2
1.6 Step-by-Step Usage	2
1.6.1 1. Convert CSV to Length-Indicated	2
1.6.2 2. Convert to Blocked Sequence Set	2
1.6.3 3. Build Primary Key Index	2
1.6.4 4. Dump the File	3
1.6.5 5. Search by ZIP Code	3
1.6.6 6. Insert Records	3
1.6.7 7. Delete Records	3
1.7 Output Example (Search)	3
1.8 Output Example (Dump)	3
1.9 Error Handling	3
1.10 Authors	3
2 Class Index	5
2.1 Class List	5
3 File Index	7
3.1 File List	7
4 Class Documentation	9
4.1 AvailBlock Struct Reference	9
4.1.1 Detailed Description	9
4.1.2 Constructor & Destructor Documentation	9
4.1.2.1 AvailBlock()	9
4.1.3 Member Data Documentation	10
4.1.3.1 blankFill	10
4.1.3.2 nextAvailRBN	10
4.1.3.3 recordCount	10
4.2 Block Struct Reference	10
4.2.1 Detailed Description	11
4.2.2 Member Data Documentation	11
4.2.2.1 nextRBN	11
4.2.2.2 prevRBN	11
4.2.2.3 recordCount	11
4.2.2.4 records	11
4.3 BlockBuffer Class Reference	11
4.3.1 Detailed Description	12

4.3.2 Member Function Documentation	12
4.3.2.1 readBlock()	12
4.3.2.2 readNextAvailRBN()	13
4.3.2.3 writeAvailBlock()	14
4.3.2.4 writeBlock()	14
4.4 CSVBuffer Class Reference	15
4.4.1 Detailed Description	16
4.4.2 Constructor & Destructor Documentation	16
4.4.2.1 CSVBuffer()	16
4.4.3 Member Function Documentation	17
4.4.3.1 buildPrimaryKeyIndex()	17
4.4.3.2 convertCSVToLengthIndicated()	17
4.4.3.3 generateStateTable()	18
4.4.3.4 getRecords()	18
4.4.3.5 loadLengthIndicatedRecords()	19
4.4.3.6 loadRecords()	19
4.4.3.7 searchByZipCodes()	19
4.4.4 Member Data Documentation	20
4.4.4.1 file	20
4.4.4.2 records	20
4.5 Field Struct Reference	21
4.5.1 Detailed Description	21
4.5.2 Member Data Documentation	21
4.5.2.1 format	21
4.5.2.2 isPrimaryKey	21
4.5.2.3 name	22
4.5.2.4 type	22
4.6 Header Class Reference	22
4.6.1 Detailed Description	23
4.6.2 Constructor & Destructor Documentation	24
4.6.2.1 Header() [1/2]	24
4.6.2.2 Header() [2/2]	24
4.6.3 Member Function Documentation	24
4.6.3.1 readFromFile()	24
4.6.3.2 writeToFile()	25
4.6.4 Member Data Documentation	25
4.6.4.1 availListHeadRBN	25
4.6.4.2 blockCount	25
4.6.4.3 blockSize	25
4.6.4.4 fields	26
4.6.4.5 fieldsPerRecord	26
4.6.4.6 fileStructureType	26

4.6.4.7 headerSize	26
4.6.4.8 indexFileName	26
4.6.4.9 indexSchema	26
4.6.4.10 minBlockCapacity	27
4.6.4.11 primaryKey	27
4.6.4.12 primaryKeyFileName	27
4.6.4.13 primaryKeyOrdinal	27
4.6.4.14 recordCount	27
4.6.4.15 recordSizeFieldLength	27
4.6.4.16 sequenceListHeadRBN	28
4.6.4.17 sizeFormatType	28
4.6.4.18 sizeInBytes	28
4.6.4.19 stale	28
4.6.4.20 version	28
4.7 HeaderBuffer3 Class Reference	29
4.7.1 Detailed Description	29
4.7.2 Member Function Documentation	29
4.7.2.1 read()	29
4.7.2.2 readField()	30
4.7.2.3 write()	31
4.7.2.4 writeField()	32
4.7.3 Member Data Documentation	33
4.7.3.1 header	33
4.8 RecordBuffer Class Reference	33
4.8.1 Detailed Description	33
4.8.2 Constructor & Destructor Documentation	33
4.8.2.1 RecordBuffer()	33
4.8.3 Member Function Documentation	34
4.8.3.1 getFields()	34
4.8.3.2 getKey()	34
4.8.4 Member Data Documentation	35
4.8.4.1 fields	35
4.9 ZipRecord Struct Reference	35
4.9.1 Detailed Description	35
4.9.2 Member Data Documentation	35
4.9.2.1 county	35
4.9.2.2 latitude	36
4.9.2.3 longitude	36
4.9.2.4 placeName	36
4.9.2.5 state	36
4.9.2.6 zipCode	36

5 File Documentation	37
5.1 BlockBuffer.cpp File Reference	37
5.1.1 Detailed Description	37
5.2 BlockBuffer.cpp	38
5.3 BlockBuffer.h File Reference	39
5.3.1 Detailed Description	39
5.4 BlockBuffer.h	40
5.5 BuildIndex.cpp File Reference	40
5.5.1 Detailed Description	41
5.5.2 Function Documentation	41
5.5.2.1 extractKey()	41
5.5.2.2 main()	41
5.6 BuildIndex.cpp	42
5.7 ConvertToBlocked.cpp File Reference	43
5.7.1 Detailed Description	43
5.7.2 Function Documentation	44
5.7.2.1 main()	44
5.8 ConvertToBlocked.cpp	44
5.9 CSVBuffer.cpp File Reference	46
5.9.1 Detailed Description	47
5.10 CSVBuffer.cpp	47
5.11 CSVBuffer.h File Reference	50
5.12 CSVBuffer.h	51
5.13 DeleteRecord.cpp File Reference	52
5.13.1 Detailed Description	52
5.13.2 Function Documentation	53
5.13.2.1 extractKey()	53
5.13.2.2 loadIndex()	53
5.13.2.3 main()	53
5.13.2.4 writeIndex()	54
5.14 DeleteRecord.cpp	54
5.15 DumpBlocked.cpp File Reference	56
5.15.1 Detailed Description	57
5.15.2 Function Documentation	57
5.15.2.1 dumpLogical()	57
5.15.2.2 dumpPhysical()	58
5.15.2.3 main()	58
5.16 DumpBlocked.cpp	59
5.17 DumpUtility.cpp File Reference	60
5.17.1 Detailed Description	60
5.17.2 Function Documentation	61
5.17.2.1 dumpByLogicalOrder()	61

5.17.2.2 dumpByPhysicalOrder()	61
5.17.2.3 printAvail()	62
5.17.2.4 printBlock()	63
5.18 DumpUtility.cpp	64
5.19 DumpUtility.h File Reference	65
5.19.1 Detailed Description	66
5.19.2 Function Documentation	66
5.19.2.1 dumpByLogicalOrder()	66
5.19.2.2 dumpByPhysicalOrder()	66
5.20 DumpUtility.h	66
5.21 HeaderAndBlockStructures.h File Reference	67
5.21.1 Detailed Description	67
5.22 HeaderAndBlockStructures.h	68
5.23 HeaderBuffer.h File Reference	68
5.23.1 Detailed Description	69
5.24 HeaderBuffer.h	70
5.25 HeaderBuffer3.cpp File Reference	71
5.25.1 Detailed Description	72
5.26 HeaderBuffer3.cpp	72
5.27 HeaderBuffer3.h File Reference	73
5.27.1 Detailed Description	74
5.28 HeaderBuffer3.h	74
5.29 InsertRecord.cpp File Reference	75
5.29.1 Detailed Description	76
5.29.2 Function Documentation	76
5.29.2.1 extractKey()	76
5.29.2.2 loadIndex()	76
5.29.2.3 main()	77
5.29.2.4 writeIndex()	78
5.30 InsertRecord.cpp	79
5.31 main.cpp File Reference	80
5.31.1 Detailed Description	81
5.31.2 Function Documentation	81
5.31.2.1 main()	81
5.32 main.cpp	83
5.33 README.md File Reference	84
5.34 RecordBuffer.cpp File Reference	84
5.34.1 Detailed Description	85
5.35 RecordBuffer.cpp	85
5.36 RecordBuffer.h File Reference	85
5.36.1 Detailed Description	86
5.37 RecordBuffer.h	87

5.38 SearchIndex.cpp File Reference	87
5.38.1 Detailed Description	88
5.38.2 Function Documentation	88
5.38.2.1 extractKey()	88
5.38.2.2 loadIndex()	88
5.38.2.3 locateBlock()	89
5.38.2.4 main()	90
5.39 SearchIndex.cpp	91
Index	93

Chapter 1

ZipCode3.0

1.1 ZIP Code Blocked Sequence Set Processor

1.2 Description

ZipCode3.0 is a C++ application that processes U.S. ZIP code data in a structured blocked sequence set format. The application supports:

- Converting CSV data into a **length-indicated** format with metadata.
- Converting that format into a **blocked sequence set file (.bss)**.
- Creating a **primary key index** of high keys per block.
- Searching for ZIP code records using an efficient block index.
- Dynamically **inserting** or **deleting** ZIP code records with logging and index updates.
- Dumping the blocked file **by physical** or **logical order**.

1.3 Features

- Converts CSV → Length-Indicated → Blocked Sequence Set (.bss)
- Generates and uses a **simple key-to-RBN index file (.idx)**
- Efficient in-place record **insertion** and **deletion**
- Includes structured **header metadata**, **record sorting**, and **record splitting**
- Supports **sequential dumps** of blocks in physical or logical order
- Fast **primary key search** using `-z#####` flags
- Log messages for **splits**, **merges**, **index rewrites**, and **failures**

1.4 Project Structure

File	Description
main.cpp	Entry point for CSV processing, summary generation, or search
CSVBuffer.cpp/h	Converts, loads, searches, and summarizes ZIP code records
HeaderBuffer.h	Defines headers for length-indicated format
HeaderBuffer3.cpp/h	Defines and manages blocked sequence set headers
BlockBuffer.cpp/h	Reads and writes blocks from/to <code>.bss</code> files
RecordBuffer.cpp/h	Unpacks record strings to extract fields or keys
ConvertToBlocked.cpp	Converts <code>.txt</code> to <code>.bss</code> with structured block layout
BuildIndex.cpp	Generates <code>key → RBN</code> index from a <code>.bss</code> file
InsertRecord.cpp	Adds records to a <code>.bss</code> file and updates the index
DeleteRecord.cpp	Removes records and updates metadata/index
DumpUtility.cpp/h	Dumps <code>.bss</code> file in physical or logical order
SearchIndex.cpp	Looks up ZIP records from a <code>.bss</code> file via <code>.idx</code>

1.5 Build Instructions

Run:

```
make
```

This compiles:

- `myProgram` – main UI for conversion and summary
- `ConvertToBlocked` – convert `.txt` to `.bss`
- `BuildIndex` – generate `.idx` from `.bss`
- `InsertRecord`, `DeleteRecord` – add/remove records
- All shared `.o` dependencies

1.6 Step-by-Step Usage

1.6.1 1. Convert CSV to Length-Indicated

```
./myProgram
# Choose option 1
# Provide CSV file (e.g. us_postal_codes.csv)
# Output: zipCodes.txt
```

1.6.2 2. Convert to Blocked Sequence Set

```
./ConvertToBlocked zipCodes.txt zipCodes.bss
```

1.6.3 3. Build Primary Key Index

```
./BuildIndex zipCodes.bss zipCodes.idx
```

1.6.4 4. Dump the File

```
./myProgram zipCodes.bss --dump-physical  
./myProgram zipCodes.bss --dump-logical
```

1.6.5 5. Search by ZIP Code

```
./myProgram zipCodes.bss -z56301 -z90210 -z00000
```

1.6.6 6. Insert Records

```
./InsertRecord zipCodes.bss zipCodes.idx new_zips.csv
```

1.6.7 7. Delete Records

```
./DeleteRecord zipCodes.bss zipCodes.idx to_delete.txt
```

1.7 Output Example (Search)

```
Found: 56301,Saint Cloud,MN,Stearns,45.555,-94.167  
Zip Code 00000 not found in data block.
```

1.8 Output Example (Dump)

```
List Head: 0  
Avail Head: -1  
0 10001 10002 10003 1  
1 10004 10005 -1
```

1.9 Error Handling

- Skips malformed lines
- Handles corrupted index or .bss files
- Detects overfilled blocks before writing
- Logs when splitting, merging, or updating header/index fails

1.10 Authors

Cha Vue, Sofia Hoffman, Zoljargal Enkhbayar, Yohannes Niguesse

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AvailBlock	Structure for a free/available block	9
Block	Structure for an active block in the blocked sequence set	10
BlockBuffer	Handles packing and unpacking of active and avail blocks in the blocked sequence set	11
CSVBuffer	Class to handle CSV data and extract structured zip code records	15
Field	Structure representing metadata for a single field in the record	21
Header	Structure representing the header of a blocked sequence set file	22
HeaderBuffer3	Class responsible for reading/writing the Header to/from a file	29
RecordBuffer	RecordBuffer class extracts fields from a comma-separated record	33
ZipRecord	Structure to store zip code information	35

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

BlockBuffer.cpp	Implements BlockBuffer class functions for handling block-level I/O	37
BlockBuffer.h	Defines the BlockBuffer class for managing read/write operations on blocked sequence set blocks	39
BuildIndex.cpp	Builds a simple primary key index file from a blocked sequence set	40
ConvertToBlocked.cpp	Converts a length-indicated file into a blocked sequence set format	43
CSVBuffer.cpp	Implementation of CSVBuffer class functions	46
CSVBuffer.h		50
DeleteRecord.cpp	Handles deletion of ZIP code records from a blocked sequence set	52
DumpBlocked.cpp	Utility to dump blocked sequence set content in physical or logical order	56
DumpUtility.cpp	Implements dump functions for the blocked sequence set structure	60
DumpUtility.h	Provides functions to dump the blocked sequence set file structure	65
HeaderAndBlockStructures.h	Defines structures for the header and block layouts used in the Blocked Sequence Set	67
HeaderBuffer.h	Defines a class for handling header records in data files	68
HeaderBuffer3.cpp	Implements methods for reading and writing blocked sequence set file headers	71
HeaderBuffer3.h	Provides a class for reading and writing header records for blocked sequence set files	73
InsertRecord.cpp	Handles insertion of new ZIP code records into a blocked sequence set	75
main.cpp	Main application to process Zip Code data from CSV or length-indicated file	80
RecordBuffer.cpp	Implements the RecordBuffer class for parsing CSV records	84
RecordBuffer.h	Provides a utility class to unpack a record string into fields	85
SearchIndex.cpp	Searches for ZIP codes in a blocked sequence set using a primary key index file	87

Chapter 4

Class Documentation

4.1 AvailBlock Struct Reference

Structure for a free/available block.

```
#include <HeaderAndBlockStructures.h>
```

Public Member Functions

- [AvailBlock](#) (int blockSize)
Constructor to initialize and blank-fill an avail block.

Public Attributes

- int [recordCount](#) = 0
Should always be 0 for an avail block.
- int [nextAvailRBN](#)
Pointer to the next available block in the avail list.
- string [blankFill](#)
Fill remaining block with whitespace for reuse.

4.1.1 Detailed Description

Structure for a free/available block.

Definition at line 51 of file [HeaderAndBlockStructures.h](#).

4.1.2 Constructor & Destructor Documentation

4.1.2.1 AvailBlock()

```
AvailBlock::AvailBlock (  
    int blockSize) [inline]
```

Constructor to initialize and blank-fill an avail block.

Definition at line 57 of file [HeaderAndBlockStructures.h](#).

4.1.3 Member Data Documentation

4.1.3.1 blankFill

```
string AvailBlock::blankFill
```

Fill remaining block with whitespace for reuse.

Definition at line 54 of file [HeaderAndBlockStructures.h](#).

4.1.3.2 nextAvailRBN

```
int AvailBlock::nextAvailRBN
```

Pointer to the next available block in the avail list.

Definition at line 53 of file [HeaderAndBlockStructures.h](#).

4.1.3.3 recordCount

```
int AvailBlock::recordCount = 0
```

Should always be 0 for an avail block.

Definition at line 52 of file [HeaderAndBlockStructures.h](#).

The documentation for this struct was generated from the following file:

- [HeaderAndBlockStructures.h](#)

4.2 Block Struct Reference

Structure for an active block in the blocked sequence set.

```
#include <HeaderAndBlockStructures.h>
```

Public Attributes

- int [recordCount](#)
Number of valid records in this block.
- int [prevRBN](#)
Relative [Block](#) Number of the previous block.
- int [nextRBN](#)
Relative [Block](#) Number of the next block.
- vector< string > [records](#)
Raw length-indicated, comma-separated record strings.

4.2.1 Detailed Description

Structure for an active block in the blocked sequence set.

Definition at line 43 of file [HeaderAndBlockStructures.h](#).

4.2.2 Member Data Documentation

4.2.2.1 nextRBN

```
int Block::nextRBN
```

Relative [Block](#) Number of the next block.

Definition at line 46 of file [HeaderAndBlockStructures.h](#).

4.2.2.2 prevRBN

```
int Block::prevRBN
```

Relative [Block](#) Number of the previous block.

Definition at line 45 of file [HeaderAndBlockStructures.h](#).

4.2.2.3 recordCount

```
int Block::recordCount
```

Number of valid records in this block.

Definition at line 44 of file [HeaderAndBlockStructures.h](#).

4.2.2.4 records

```
vector<string> Block::records
```

Raw length-indicated, comma-separated record strings.

Definition at line 47 of file [HeaderAndBlockStructures.h](#).

The documentation for this struct was generated from the following file:

- [HeaderAndBlockStructures.h](#)

4.3 BlockBuffer Class Reference

Handles packing and unpacking of active and avail blocks in the blocked sequence set.

```
#include <BlockBuffer.h>
```

Static Public Member Functions

- static bool [writeBlock](#) (std::ostream &out, const [Block](#) &block, int rbn, int blockSize)
Writes a block to the output stream at the specified RBN.
- static bool [readBlock](#) (std::istream &in, [Block](#) &block, int rbn, int blockSize)
Reads a block from the input stream at the specified RBN.
- static void [writeAvailBlock](#) (std::ostream &out, const [AvailBlock](#) &avail, int rbn, int blockSize)
Writes an avail block to the output stream at the specified RBN.
- static int [readNextAvailRBN](#) (std::ifstream &in, int rbn, int blockSize)
Reads the RBN of the next avail block (for traversing avail list).

4.3.1 Detailed Description

Handles packing and unpacking of active and avail blocks in the blocked sequence set.

Definition at line 11 of file [BlockBuffer.h](#).

4.3.2 Member Function Documentation

4.3.2.1 readBlock()

```
static bool BlockBuffer::readBlock (
    std::istream & in,
    Block & block,
    int rbn,
    int blockSize) [static]
```

Reads a block from the input stream at the specified RBN.

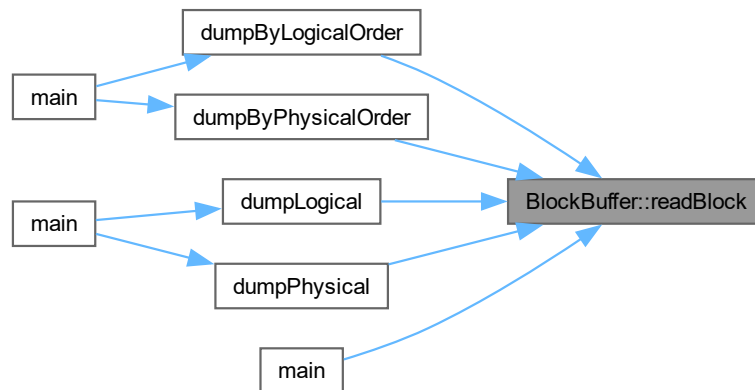
Parameters

<i>in</i>	Input stream (binary mode).
<i>block</i>	Destination block structure to populate.
<i>rbn</i>	The relative block number (RBN) to read from.
<i>blockSize</i>	The fixed size of each block in bytes.

Returns

True if read is successful.

Here is the caller graph for this function:

**4.3.2.2 readNextAvailRBN()**

```

static int BlockBuffer::readNextAvailRBN (
    std::ifstream & in,
    int rbn,
    int blockSize) [static]

```

Reads the RBN of the next avail block (for traversing avail list).

Parameters

<i>in</i>	Input file stream (binary mode).
<i>rbn</i>	The current avail block RBN.
<i>blockSize</i>	The fixed block size.

Returns

RBN of next avail block or -1 on error.

Here is the caller graph for this function:



4.3.2.3 writeAvailBlock()

```
static void BlockBuffer::writeAvailBlock (
    std::ostream & out,
    const AvailBlock & avail,
    int rbn,
    int blockSize) [static]
```

Writes an avail block to the output stream at the specified RBN.

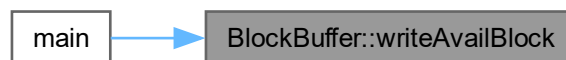
Parameters

<i>out</i>	Output file stream (binary mode).
<i>availBlock</i>	The avail block to write.
<i>rbn</i>	The RBN to write at.
<i>blockSize</i>	The fixed size of each block in bytes.

Returns

True if write is successful.

Here is the caller graph for this function:



4.3.2.4 writeBlock()

```
bool BlockBuffer::writeBlock (
    std::ostream & out,
    const Block & block,
    int rbn,
    int blockSize) [static]
```

Writes a block to the output stream at the specified RBN.

Parameters

<i>out</i>	Output file stream (binary mode).
<i>block</i>	The block structure to write.
<i>rbn</i>	The relative block number (RBN) to write at.
<i>blockSize</i>	The fixed size of each block in bytes.

Returns

True if write is successful.

Definition at line 9 of file [BlockBuffer.cpp](#).

Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- [BlockBuffer.h](#)
- [BlockBuffer.cpp](#)

4.4 CSVBuffer Class Reference

Class to handle CSV data and extract structured zip code records.

```
#include <CSVBuffer.h>
```

Public Member Functions

- [CSVBuffer](#) (const string &filename)
Constructor that opens a CSV or length-indicated file.
- const vector< [ZipRecord](#) > & [getRecords](#) () const
Retrieves all zip code records.
- void [generateStateTable](#) () const
Generates a state-wise summary of extreme zip codes.
- unordered_map< int, [ZipRecord](#) > [buildPrimaryKeyIndex](#) () const
Builds an in-memory primary key index of zip codes.
- void [searchByZipCodes](#) (const vector< int > &zipCodes) const
Searches and prints zip code records from a list.

Static Public Member Functions

- static void [convertCSVToLengthIndicated](#) (const string &inputFile, const string &outputFile)
Converts a standard CSV file to a length-indicated format.

Private Member Functions

- void [loadRecords](#) ()
Loads records from the CSV file into memory.
- void [loadLengthIndicatedRecords](#) ()
Loads length-indicated records from a file.

Private Attributes

- ifstream [file](#)
File stream to read CSV data.
- vector< [ZipRecord](#) > [records](#)
Vector to store all zip code records.

4.4.1 Detailed Description

Class to handle CSV data and extract structured zip code records.

Definition at line 29 of file [CSVBuffer.h](#).

4.4.2 Constructor & Destructor Documentation

4.4.2.1 CSVBuffer()

```
CSVBuffer::CSVBuffer (
    const string & filename)
```

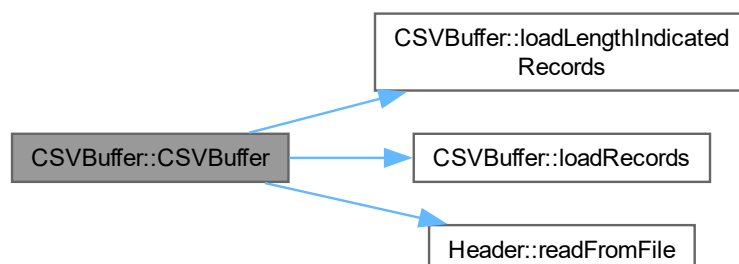
Constructor that opens a CSV or length-indicated file.

Parameters

<i>filename</i>	Name of the input file.
-----------------	-------------------------

Definition at line 19 of file [CSVBuffer.cpp](#).

Here is the call graph for this function:



4.4.3 Member Function Documentation

4.4.3.1 buildPrimaryKeyIndex()

```
unordered_map< int, ZipRecord > CSVBuffer::buildPrimaryKeyIndex () const
```

Builds an in-memory primary key index of zip codes.

Returns

An unordered map from zip code to [ZipRecord](#).

Definition at line 34 of file [CSVBuffer.cpp](#).

Here is the caller graph for this function:



4.4.3.2 convertCSVToLengthIndicated()

```
void CSVBuffer::convertCSVToLengthIndicated (
    const string & inputFile,
    const string & outputFile) [static]
```

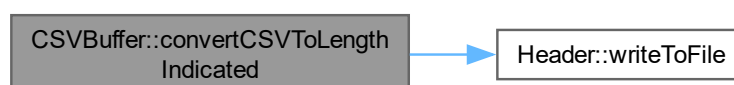
Converts a standard CSV file to a length-indicated format.

Parameters

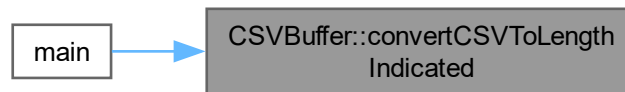
<i>inputFile</i>	The input CSV filename.
<i>outputFile</i>	The output filename in length-indicated format.

Definition at line 108 of file [CSVBuffer.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.3.3 generateStateTable()

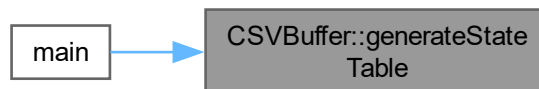
```
void CSVBuffer::generateStateTable () const
```

Generates a state-wise summary of extreme zip codes.

Finds and outputs the easternmost, westernmost, northernmost, and southernmost zip codes per state.

Definition at line 189 of file [CSVBuffer.cpp](#).

Here is the caller graph for this function:



4.4.3.4 getRecords()

```
const vector< ZipRecord > & CSVBuffer::getRecords () const
```

Retrieves all zip code records.

Returns

A constant reference to a vector containing all [ZipRecord](#) structures.

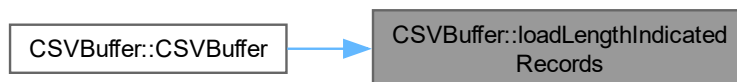
4.4.3.5 loadLengthIndicatedRecords()

```
void CSVBuffer::loadLengthIndicatedRecords () [private]
```

Loads length-indicated records from a file.

Definition at line 81 of file [CSVBuffer.cpp](#).

Here is the caller graph for this function:



4.4.3.6 loadRecords()

```
void CSVBuffer::loadRecords () [private]
```

Loads records from the CSV file into memory.

Definition at line 57 of file [CSVBuffer.cpp](#).

Here is the caller graph for this function:



4.4.3.7 searchByZipCodes()

```
void CSVBuffer::searchByZipCodes (  
    const vector< int > & zipCodes) const
```

Searches and prints zip code records from a list.

Parameters

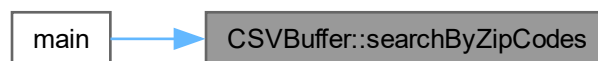
<i>zipCodes</i>	Vector of zip codes to look up.
-----------------	---------------------------------

Definition at line 42 of file [CSVBuffer.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.4 Member Data Documentation

4.4.4.1 file

```
ifstream CSVBuffer::file [private]
```

File stream to read CSV data.

Definition at line 31 of file [CSVBuffer.h](#).

4.4.4.2 records

```
vector<ZipRecord> CSVBuffer::records [private]
```

Vector to store all zip code records.

Definition at line 32 of file [CSVBuffer.h](#).

The documentation for this class was generated from the following files:

- [CSVBuffer.h](#)
- [CSVBuffer.cpp](#)

4.5 Field Struct Reference

Structure representing metadata for a single field in the record.

```
#include <HeaderAndBlockStructures.h>
```

Public Attributes

- string [name](#)
Field name or ID.
- string [type](#)
Field data type.
- string [format](#)
Format description (e.g. CSV, text).
- bool [isPrimaryKey](#) = false
True if this field is the primary key.

4.5.1 Detailed Description

Structure representing metadata for a single field in the record.

Structure to represent a field in the header.

Definition at line 14 of file [HeaderAndBlockStructures.h](#).

4.5.2 Member Data Documentation

4.5.2.1 format

```
string Field::format
```

Format description (e.g. CSV, text).

Format of the field data.

Definition at line 17 of file [HeaderAndBlockStructures.h](#).

4.5.2.2 isPrimaryKey

```
bool Field::isPrimaryKey = false
```

True if this field is the primary key.

Indicates if this field is the primary key.

Definition at line 18 of file [HeaderAndBlockStructures.h](#).

4.5.2.3 name

```
string Field::name
```

[Field](#) name or ID.

[Field](#) name.

Definition at line 15 of file [HeaderAndBlockStructures.h](#).

4.5.2.4 type

```
string Field::type
```

[Field](#) data type.

Data type of the field.

Definition at line 16 of file [HeaderAndBlockStructures.h](#).

The documentation for this struct was generated from the following files:

- [HeaderAndBlockStructures.h](#)
- [HeaderBuffer.h](#)

4.6 Header Class Reference

Structure representing the header of a blocked sequence set file.

```
#include <HeaderAndBlockStructures.h>
```

Public Member Functions

- [Header](#) ()
Default constructor initializing fields to default values.
- [Header](#) (const string &[fileStructureType](#), const string &[version](#), int [sizeInBytes](#), const string &[sizeFormatType](#), const string &[primaryKeyFileName](#), int [recordCount](#), const vector< [Field](#) > &[fields](#), const string &[primaryKey](#))
Parameterized constructor to initialize with custom values.
- void [writeToFile](#) (ofstream &out) const
Writes the header to a file.
- bool [readFromFile](#) (ifstream &in)
Reads the header from a file.

Public Attributes

- string [fileStructureType](#)
e.g., "BlockedSequenceSet"
- string [version](#)
Version of the format.
- int [headerSize](#)
Total size in bytes of the header.
- int [recordSizeFieldLength](#)
Size in bytes of the length field for each record.
- string [sizeFormatType](#)
e.g., "ASCII" or "binary"
- int [blockSize](#)
Size of each block in bytes (e.g., 512).
- float [minBlockCapacity](#)
Minimum block fill ratio (e.g., 0.5).
- string [indexFileName](#)
Associated index file.
- string [indexSchema](#)
Description of index format.
- int [recordCount](#)
Total number of records.
- int [blockCount](#)
Total number of blocks.
- int [fieldsPerRecord](#)
Number of fields in each record.
- vector< [Field](#) > [fields](#)
Field metadata list.
- int [primaryKeyOrdinal](#)
Index of primary key field in the fields vector.
- int [availListHeadRBN](#)
Head of the avail list (RBN).
- int [sequenceListHeadRBN](#)
Head of the active block list (RBN).
- bool [stale](#)
Indicates if the data is stale.
- int [sizeInBytes](#)
- string [primaryKeyFileName](#)
- string [primaryKey](#)

4.6.1 Detailed Description

Structure representing the header of a blocked sequence set file.

Class to manage header metadata for record-based file storage.

Definition at line 22 of file [HeaderAndBlockStructures.h](#).

4.6.2 Constructor & Destructor Documentation

4.6.2.1 Header() [1/2]

```
Header::Header () [inline]
```

Default constructor initializing fields to default values.

Definition at line 40 of file [HeaderBuffer.h](#).

4.6.2.2 Header() [2/2]

```
Header::Header (
    const string & fileStructureType,
    const string & version,
    int sizeInBytes,
    const string & sizeFormatType,
    const string & primaryKeyFileName,
    int recordCount,
    const vector< Field > & fields,
    const string & primaryKey) [inline]
```

Parameterized constructor to initialize with custom values.

Definition at line 45 of file [HeaderBuffer.h](#).

4.6.3 Member Function Documentation

4.6.3.1 readFromFile()

```
bool Header::readFromFile (
    ifstream & in) [inline]
```

Reads the header from a file.

Parameters

<i>in</i>	Input file stream to read the structured header.
-----------	--

Returns

True if successfully read, false otherwise.

Definition at line 77 of file [HeaderBuffer.h](#).

Here is the caller graph for this function:



4.6.3.2 writeToFile()

```
void Header::writeToFile (
    ofstream & out) const [inline]
```

Writes the header to a file.

Parameters

<i>out</i>	Output file stream to write the structured header.
------------	--

Definition at line 58 of file [HeaderBuffer.h](#).

Here is the caller graph for this function:



4.6.4 Member Data Documentation

4.6.4.1 availListHeadRBN

```
int Header::availListHeadRBN
```

Head of the avail list (RBN).

Definition at line 37 of file [HeaderAndBlockStructures.h](#).

4.6.4.2 blockCount

```
int Header::blockCount
```

Total number of blocks.

Definition at line 33 of file [HeaderAndBlockStructures.h](#).

4.6.4.3 blockSize

```
int Header::blockSize
```

Size of each block in bytes (e.g., 512).

Definition at line 28 of file [HeaderAndBlockStructures.h](#).

4.6.4.4 fields

```
vector< Field > Header::fields
```

[Field](#) metadata list.

Definition at line 35 of file [HeaderAndBlockStructures.h](#).

4.6.4.5 fieldsPerRecord

```
int Header::fieldsPerRecord
```

Number of fields in each record.

Definition at line 34 of file [HeaderAndBlockStructures.h](#).

4.6.4.6 fileStructureType

```
string Header::fileStructureType
```

e.g., "BlockedSequenceSet"

Definition at line 23 of file [HeaderAndBlockStructures.h](#).

4.6.4.7 headerSize

```
int Header::headerSize
```

Total size in bytes of the header.

Definition at line 25 of file [HeaderAndBlockStructures.h](#).

4.6.4.8 indexFileName

```
string Header::indexFileName
```

Associated index file.

Definition at line 30 of file [HeaderAndBlockStructures.h](#).

4.6.4.9 indexSchema

```
string Header::indexSchema
```

Description of index format.

Definition at line 31 of file [HeaderAndBlockStructures.h](#).

4.6.4.10 minBlockCapacity

```
float Header::minBlockCapacity
```

Minimum block fill ratio (e.g., 0.5).

Definition at line 29 of file [HeaderAndBlockStructures.h](#).

4.6.4.11 primaryKey

```
string Header::primaryKey
```

Definition at line 37 of file [HeaderBuffer.h](#).

4.6.4.12 primaryKeyFileName

```
string Header::primaryKeyFileName
```

Definition at line 33 of file [HeaderBuffer.h](#).

4.6.4.13 primaryKeyOrdinal

```
int Header::primaryKeyOrdinal
```

Index of primary key field in the fields vector.

Definition at line 36 of file [HeaderAndBlockStructures.h](#).

4.6.4.14 recordCount

```
int Header::recordCount
```

Total number of records.

Definition at line 32 of file [HeaderAndBlockStructures.h](#).

4.6.4.15 recordSizeFieldLength

```
int Header::recordSizeFieldLength
```

Size in bytes of the length field for each record.

Definition at line 26 of file [HeaderAndBlockStructures.h](#).

4.6.4.16 sequenceListHeadRBN

```
int Header::sequenceListHeadRBN
```

Head of the active block list (RBN).

Definition at line 38 of file [HeaderAndBlockStructures.h](#).

4.6.4.17 sizeFormatType

```
string Header::sizeFormatType
```

e.g., "ASCII" or "binary"

Definition at line 27 of file [HeaderAndBlockStructures.h](#).

4.6.4.18 sizeInBytes

```
int Header::sizeInBytes
```

Definition at line 31 of file [HeaderBuffer.h](#).

4.6.4.19 stale

```
bool Header::stale
```

Indicates if the data is stale.

Definition at line 39 of file [HeaderAndBlockStructures.h](#).

4.6.4.20 version

```
string Header::version
```

Version of the format.

Definition at line 24 of file [HeaderAndBlockStructures.h](#).

The documentation for this class was generated from the following files:

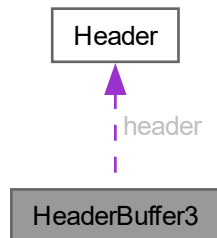
- [HeaderAndBlockStructures.h](#)
- [HeaderBuffer.h](#)

4.7 HeaderBuffer3 Class Reference

Class responsible for reading/writing the [Header](#) to/from a file.

```
#include <HeaderBuffer3.h>
```

Collaboration diagram for HeaderBuffer3:



Public Member Functions

- bool [read](#) (istream &in)
Reads a header record from the input stream.
- void [write](#) (ostream &out) const
Writes the header record to the output stream.

Public Attributes

- [Header](#) [header](#)
The header structure being managed.

Private Member Functions

- [Field](#) [readField](#) (istream &in)
Helper to read a single field definition from stream.
- void [writeField](#) (ostream &out, const [Field](#) &field) const
Helper to write a single field definition to stream.

4.7.1 Detailed Description

Class responsible for reading/writing the [Header](#) to/from a file.

Definition at line 13 of file [HeaderBuffer3.h](#).

4.7.2 Member Function Documentation

4.7.2.1 read()

```
bool HeaderBuffer3::read (
    istream & in)
```

Reads a header record from the input stream.

Reads a header record from the given input stream.

Parameters

<i>in</i>	Reference to the input stream (e.g. ifstream or fstream).
-----------	---

Returns

True if read successfully, false otherwise.

Parameters

<i>in</i>	The input stream to read the header from.
-----------	---

Returns

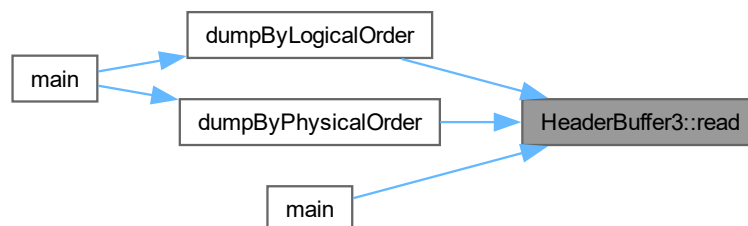
true if the header was successfully read, false otherwise.

Definition at line 14 of file [HeaderBuffer3.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**4.7.2.2 readField()**

```
Field HeaderBuffer3::readField (  
    istream & in) [private]
```

Helper to read a single field definition from stream.

Reads a single [Field](#) structure from the stream.

Parameters

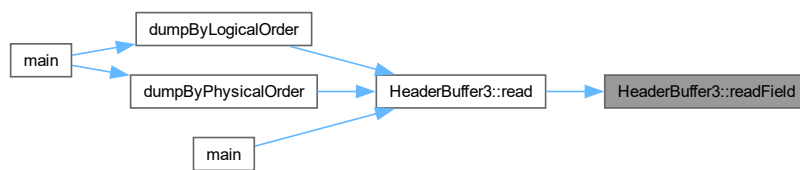
<i>in</i>	The input stream to read from.
-----------	--------------------------------

Returns

A [Field](#) populated with values read from the stream.

Definition at line 89 of file [HeaderBuffer3.cpp](#).

Here is the caller graph for this function:



4.7.2.3 write()

```
void HeaderBuffer3::write (
    ostream & out) const
```

Writes the header record to the output stream.

Writes the header record to the given output stream.

Parameters

<i>out</i>	Reference to the output file stream.
------------	--------------------------------------

Returns

True if write successful, false otherwise.

Parameters

<i>out</i>	The output file stream to write the header to.
------------	--

Definition at line 55 of file [HeaderBuffer3.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.4 writeField()

```

void HeaderBuffer3::writeField (
    ostream & out,
    const Field & field) const [private]
  
```

Helper to write a single field definition to stream.

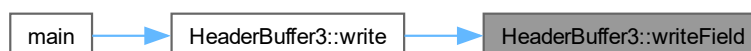
Writes a single [Field](#) structure to the stream.

Parameters

<i>out</i>	The output stream.
<i>field</i>	The Field to write.

Definition at line [104](#) of file [HeaderBuffer3.cpp](#).

Here is the caller graph for this function:



4.7.3 Member Data Documentation

4.7.3.1 header

[Header](#) `HeaderBuffer3::header`

The header structure being managed.

Definition at line 15 of file [HeaderBuffer3.h](#).

The documentation for this class was generated from the following files:

- [HeaderBuffer3.h](#)
- [HeaderBuffer3.cpp](#)

4.8 RecordBuffer Class Reference

[RecordBuffer](#) class extracts fields from a comma-separated record.

```
#include <RecordBuffer.h>
```

Public Member Functions

- [RecordBuffer](#) (const std::string &record)
Constructs a [RecordBuffer](#) with a record string.
- std::string [getKey](#) () const
Returns the value of the key field (assumed to be first).
- std::vector< std::string > [getFields](#) () const
Returns all fields unpacked from the record.

Private Attributes

- std::vector< std::string > [fields](#)

4.8.1 Detailed Description

[RecordBuffer](#) class extracts fields from a comma-separated record.

Definition at line 11 of file [RecordBuffer.h](#).

4.8.2 Constructor & Destructor Documentation

4.8.2.1 RecordBuffer()

```
RecordBuffer::RecordBuffer (  
    const std::string & record)
```

Constructs a [RecordBuffer](#) with a record string.

Parameters

<i>record</i>	Comma-separated field values.
---------------	-------------------------------

Definition at line 7 of file [RecordBuffer.cpp](#).

4.8.3 Member Function Documentation

4.8.3.1 getFields()

```
std::vector< std::string > RecordBuffer::getFields () const
```

Returns all fields unpacked from the record.

Returns

Vector of string fields.

Definition at line 19 of file [RecordBuffer.cpp](#).

4.8.3.2 getKey()

```
std::string RecordBuffer::getKey () const
```

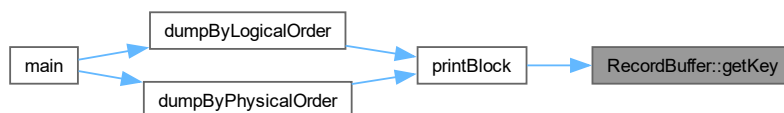
Returns the value of the key field (assumed to be first).

Returns

Key field as a string.

Definition at line 15 of file [RecordBuffer.cpp](#).

Here is the caller graph for this function:



4.8.4 Member Data Documentation

4.8.4.1 fields

```
std::vector<std::string> RecordBuffer::fields [private]
```

Definition at line 26 of file [RecordBuffer.h](#).

The documentation for this class was generated from the following files:

- [RecordBuffer.h](#)
- [RecordBuffer.cpp](#)

4.9 ZipRecord Struct Reference

Structure to store zip code information.

```
#include <CSVBuffer.h>
```

Public Attributes

- int [zipCode](#)
Zip code number.
- string [placeName](#)
Name of the place.
- string [state](#)
Two-letter state abbreviation.
- string [county](#)
County name.
- double [latitude](#)
Latitude coordinate.
- double [longitude](#)
Longitude coordinate.

4.9.1 Detailed Description

Structure to store zip code information.

Definition at line 19 of file [CSVBuffer.h](#).

4.9.2 Member Data Documentation

4.9.2.1 county

```
string ZipRecord::county
```

County name.

Definition at line 23 of file [CSVBuffer.h](#).

4.9.2.2 latitude

```
double ZipRecord::latitude
```

Latitude coordinate.

Definition at line 24 of file [CSVBuffer.h](#).

4.9.2.3 longitude

```
double ZipRecord::longitude
```

Longitude coordinate.

Definition at line 25 of file [CSVBuffer.h](#).

4.9.2.4 placeName

```
string ZipRecord::placeName
```

Name of the place.

Definition at line 21 of file [CSVBuffer.h](#).

4.9.2.5 state

```
string ZipRecord::state
```

Two-letter state abbreviation.

Definition at line 22 of file [CSVBuffer.h](#).

4.9.2.6 zipCode

```
int ZipRecord::zipCode
```

Zip code number.

Definition at line 20 of file [CSVBuffer.h](#).

The documentation for this struct was generated from the following file:

- [CSVBuffer.h](#)

Chapter 5

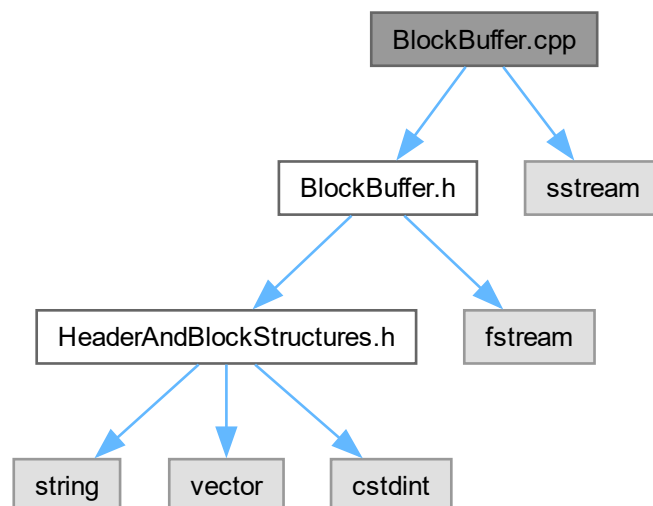
File Documentation

5.1 BlockBuffer.cpp File Reference

Implements [BlockBuffer](#) class functions for handling block-level I/O.

```
#include "BlockBuffer.h"  
#include <sstream>
```

Include dependency graph for BlockBuffer.cpp:



5.1.1 Detailed Description

Implements [BlockBuffer](#) class functions for handling block-level I/O.

Definition in file [BlockBuffer.cpp](#).

5.2 BlockBuffer.cpp

[Go to the documentation of this file.](#)

```

00001
00002
00003
00004 #include "BlockBuffer.h"
00005 #include <sstream>
00006
00007 using namespace std;
00008
00009 bool BlockBuffer::writeBlock(std::ostream& out, const Block& block, int rbn, int blockSize) {
00010     if (out.fail()) return false;
00011     streampos pos = static_cast<streampos>(rbn) * blockSize;
00012     out.seekp(pos);
00013
00014     stringstream buffer;
00015     buffer << block.recordCount << '\n';
00016     buffer << block.prevRBN << '\n';
00017     buffer << block.nextRBN << '\n';
00018     for (const auto& rec : block.records) {
00019         buffer << rec << '\n';
00020     }
00021
00022     string rawBlock = buffer.str();
00023     if (rawBlock.size() > static_cast<size_t>(blockSize)) return false;
00024     rawBlock.resize(blockSize, ' ');
00025     out.write(rawBlock.c_str(), blockSize);
00026     return true;
00027 }
00028
00029 bool BlockBuffer::readBlock(istream& in, Block& block, int rbn, int blockSize) {
00030     if (!in) return false;
00031     streampos pos = static_cast<streampos>(rbn) * blockSize;
00032     in.seekg(pos);
00033
00034     vector<char> buffer(blockSize);
00035     in.read(buffer.data(), blockSize);
00036     stringstream ss(string(buffer.data(), blockSize));
00037
00038     ss >> block.recordCount;
00039     ss.ignore();
00040     ss >> block.prevRBN;
00041     ss.ignore();
00042     ss >> block.nextRBN;
00043     ss.ignore();
00044
00045     block.records.clear();
00046     string line;
00047     for (int i = 0; i < block.recordCount && getline(ss, line); ++i) {
00048         block.records.push_back(line);
00049     }
00050
00051     return true;
00052 }
00053
00054 void BlockBuffer::writeAvailBlock(ostream& out, const AvailBlock& avail, int rbn, int blockSize) {
00055     streampos pos = static_cast<streampos>(rbn) * blockSize;
00056     out.seekp(pos);
00057
00058     stringstream buffer;
00059     buffer << avail.recordCount << '\n';
00060     buffer << avail.nextAvailRBN << '\n';
00061     string data = buffer.str();
00062     data += avail.blankFill;
00063     data.resize(blockSize, ' ');
00064
00065     out.write(data.c_str(), blockSize);
00066 }
00067
00068 int BlockBuffer::readNextAvailRBN(istream& in, int rbn, int blockSize) {
00069     if (!in.is_open()) return -1;
00070     streampos pos = static_cast<streampos>(rbn) * blockSize;
00071     in.seekg(pos);
00072
00073     int recordCount;
00074     int nextRBN;
00075     in >> recordCount;
00076     in.ignore();
00077     in >> nextRBN;
00078     return nextRBN;
00079 }

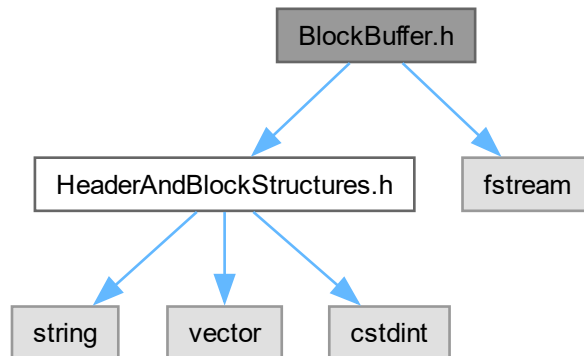
```

5.3 BlockBuffer.h File Reference

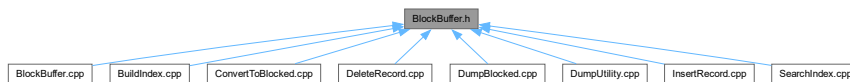
Defines the [BlockBuffer](#) class for managing read/write operations on blocked sequence set blocks.

```
#include "HeaderAndBlockStructures.h"  
#include <fstream>
```

Include dependency graph for BlockBuffer.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [BlockBuffer](#)

Handles packing and unpacking of active and avail blocks in the blocked sequence set.

5.3.1 Detailed Description

Defines the [BlockBuffer](#) class for managing read/write operations on blocked sequence set blocks.

Definition in file [BlockBuffer.h](#).

5.4 BlockBuffer.h

[Go to the documentation of this file.](#)

```

00001
00003
00004 #ifndef BLOCKBUFFER_H
00005 #define BLOCKBUFFER_H
00006
00007 #include "HeaderAndBlockStructures.h"
00008 #include <fstream>
00009
00011 class BlockBuffer {
00012 public:
00019     static bool writeBlock(std::ostream& out, const Block& block, int rbn, int blockSize);
00020
00027     static bool readBlock(std::istream& in, Block& block, int rbn, int blockSize);
00028
00035     static void writeAvailBlock(std::ostream& out, const AvailBlock& avail, int rbn, int blockSize);
00036
00042     static int readNextAvailRBN(std::ifstream& in, int rbn, int blockSize);
00043 };
00044
00045 #endif // BLOCKBUFFER_H

```

5.5 BuildIndex.cpp File Reference

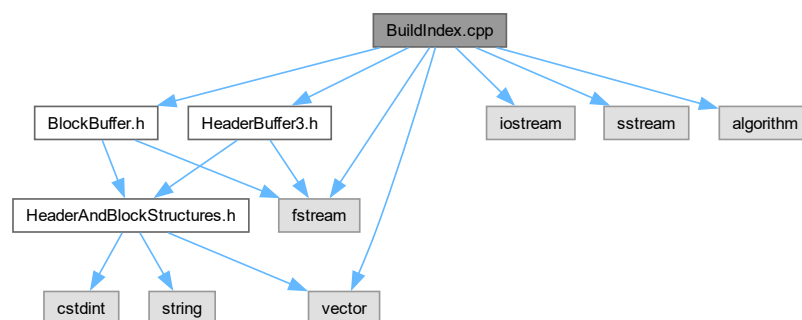
Builds a simple primary key index file from a blocked sequence set.

```

#include "HeaderBuffer3.h"
#include "BlockBuffer.h"
#include <fstream>
#include <iostream>
#include <vector>
#include <sstream>
#include <algorithm>

```

Include dependency graph for BuildIndex.cpp:



Functions

- string [extractKey](#) (const string &record)
Extracts the primary key (ZIP code) from a record string.
- int [main](#) (int argc, char *argv[])
Entry point to build an index file from a blocked sequence set.

5.5.1 Detailed Description

Builds a simple primary key index file from a blocked sequence set.

Definition in file [BuildIndex.cpp](#).

5.5.2 Function Documentation

5.5.2.1 `extractKey()`

```
string extractKey (  
    const string & record)
```

Extracts the primary key (ZIP code) from a record string.

Definition at line 15 of file [BuildIndex.cpp](#).

Here is the caller graph for this function:



5.5.2.2 `main()`

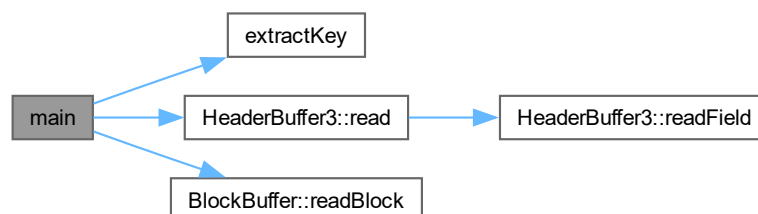
```
int main (  
    int argc,  
    char * argv[])
```

Entry point to build an index file from a blocked sequence set.

< Vector of {highest key in block, RBN}

Definition at line 23 of file [BuildIndex.cpp](#).

Here is the call graph for this function:



5.6 BuildIndex.cpp

[Go to the documentation of this file.](#)

```

00001
00002
00003
00004 #include "HeaderBuffer3.h"
00005 #include "BlockBuffer.h"
00006 #include <fstream>
00007 #include <iostream>
00008 #include <vector>
00009 #include <sstream>
00010 #include <algorithm>
00011
00012 using namespace std;
00013
00014 string extractKey(const string& record) {
00015     stringstream ss(record);
00016     string key;
00017     getline(ss, key, ',');
00018     return key;
00019 }
00020
00021 int main(int argc, char* argv[]) {
00022     if (argc < 3) {
00023         cerr << "Usage: " << argv[0] << " <blocked_file> <output_index_file>" << endl;
00024         return 1;
00025     }
00026
00027     string blockedFile = argv[1];
00028     string indexFile = argv[2];
00029
00030     ifstream in(blockedFile, ios::binary);
00031     if (!in.is_open()) {
00032         cerr << "Error opening blocked file." << endl;
00033         return 1;
00034     }
00035
00036     HeaderBuffer3 hb;
00037     if (!hb.read(in)) {
00038         cerr << "Failed to read header." << endl;
00039         return 1;
00040     }
00041
00042     int blockSize = hb.header.blockSize;
00043     int blockCount = hb.header.blockCount;
00044
00045     cout << "[Debug] Block Size: " << blockSize << endl;
00046     cout << "[Debug] Block Count: " << blockCount << endl;
00047
00048     if (blockSize <= 0 || blockCount <= 0) {
00049         cerr << "Invalid block size or block count in header." << endl;
00050         return 1;
00051     }
00052
00053     streampos startOfBlocks = in.tellg();
00054     vector<pair<string, int>> index;
00055
00056     for (int rbn = 0; rbn < blockCount; ++rbn) {
00057         in.clear();
00058         in.seekg(startOfBlocks + streampos(rbn * blockSize));
00059
00060         Block blk;
00061         if (!BlockBuffer::readBlock(in, blk, rbn, blockSize)) {
00062             cerr << "Warning: Could not read block " << rbn << endl;
00063             continue;
00064         }
00065
00066         if (blk.recordCount == 0 || blk.records.empty()) continue;
00067
00068         vector<string> keys;
00069         for (const auto& rec : blk.records) {
00070             if (!rec.empty()) {
00071                 keys.push_back(extractKey(rec));
00072             }
00073         }
00074
00075         if (keys.empty()) continue;
00076
00077         sort(keys.begin(), keys.end());
00078         string maxKey = keys.back();
00079         index.emplace_back(maxKey, rbn);
00080     }
00081
00082     in.close();
00083     ofstream out(indexFile);

```

```

00086     if (!out.is_open()) {
00087         cerr << "Error opening output index file." << endl;
00088         return 1;
00089     }
00090
00091     for (const auto& [key, rbn] : index) {
00092         out << key << "," << rbn << endl;
00093     }
00094
00095     cout << "Index file generated: " << indexFile << endl;
00096     return 0;
00097 }

```

5.7 ConvertToBlocked.cpp File Reference

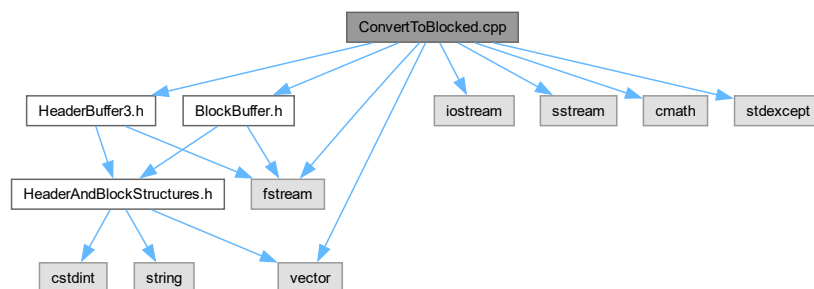
Converts a length-indicated file into a blocked sequence set format.

```

#include "HeaderBuffer3.h"
#include "BlockBuffer.h"
#include <fstream>
#include <iostream>
#include <vector>
#include <sstream>
#include <cmath>
#include <stdexcept>

```

Include dependency graph for ConvertToBlocked.cpp:



Functions

- `int main (int argc, char *argv[ ])`
Entry point for the CSV-to-blocked conversion tool.

5.7.1 Detailed Description

Converts a length-indicated file into a blocked sequence set format.

Definition in file [ConvertToBlocked.cpp](#).

5.7.2 Function Documentation

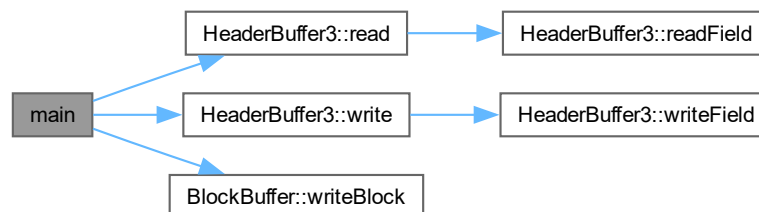
5.7.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Entry point for the CSV-to-blocked conversion tool.

Definition at line 16 of file [ConvertToBlocked.cpp](#).

Here is the call graph for this function:



5.8 ConvertToBlocked.cpp

[Go to the documentation of this file.](#)

```

00001
00002
00003
00004 #include "HeaderBuffer3.h"
00005 #include "BlockBuffer.h"
00006 #include <fstream>
00007 #include <iostream>
00008 #include <vector>
00009 #include <sstream>
00010 #include <cmath>
00011 #include <stdexcept>
00012
00013 using namespace std;
00014
00015 int main(int argc, char* argv[]) {
00016     const int MAX_BLOCKS = 50000; // Cap to prevent runaway memory
00017
00018     if (argc < 3) {
00019         cerr << "Usage: " << argv[0] << " <input_length_indicated_file> <output_blocked_file>" << endl;
00020         return 1;
00021     }
00022
00023     string inputFile = argv[1];
00024     string outputFile = argv[2];
00025
00026     ifstream in(inputFile);
00027     if (!in.is_open()) {
00028         cerr << "Error: Could not open input file." << endl;
00029         return 1;
00030     }
00031
00032     fstream out(outputFile, ios::in | ios::out | ios::binary | ios::trunc);
00033     if (!out.is_open()) {
00034         cerr << "Error: Could not create output file." << endl;
00035         return 1;
00036     }
00037 }
```

```

00038
00039     HeaderBuffer3 hb;
00040     if (!hb.read(in)) {
00041         cerr << "Error: Failed to read header from input file." << endl;
00042         return 1;
00043     }
00044
00045     int blockSize = 1024 * 4; // 4 KB blocks
00046     if (hb.header.blockSize > 0) {
00047         blockSize = hb.header.blockSize;
00048     }
00049     hb.header.blockSize = blockSize;
00050     if (blockSize < 512) {
00051         cerr << "Error: Block size must be at least 512 bytes." << endl;
00052         return 1;
00053     }
00054
00055     vector<string> buffer;
00056     string line;
00057     int currentRBN = 0;
00058     int prevRBN = -1;
00059     int blockCount = 0;
00060     int sequenceListHeadRBN = -1;
00061
00062     stringstream dummyHeader;
00063     hb.write(dummyHeader);
00064     string headerText = dummyHeader.str();
00065
00066     if (headerText.size() > static_cast<size_t>(blockSize)) {
00067         cerr << "Error: Header size exceeds block size." << endl;
00068         return 1;
00069     }
00070
00071     string paddedHeader = headerText;
00072     paddedHeader.resize(blockSize, ' ');
00073
00074     out.seekp(0);
00075     out.write(paddedHeader.c_str(), blockSize);
00076
00077     int totalHeaderLines = 1 + hb.header.fieldsPerRecord;
00078     for (int i = 0; i < totalHeaderLines; ++i) {
00079         if (!getline(in, line)) {
00080             cerr << "Error: Failed to skip header lines." << endl;
00081             return 1;
00082         }
00083     }
00084
00085     size_t currentSize = 0;
00086     int lineCount = 0;
00087     while (getline(in, line)) {
00088         ++lineCount;
00089         if (line.empty()) continue;
00090
00091         if (line.find(',') == string::npos) {
00092             cerr << "[Error] Skipping malformed line: " << line << endl;
00093             continue;
00094         }
00095
00096         size_t lineSize = line.size() + 1;
00097         if (lineSize > static_cast<size_t>(blockSize)) {
00098             cerr << "Error: Line too large for block." << endl;
00099             return 1;
00100         }
00101
00102         if (currentSize + lineSize > static_cast<size_t>(blockSize)) {
00103             if (blockCount >= MAX_BLOCKS) {
00104                 cerr << "Error: Max block count exceeded (" << MAX_BLOCKS << "). Aborting." << endl;
00105                 return 1;
00106             }
00107
00108             Block blk;
00109             blk.recordCount = buffer.size();
00110             blk.records = buffer;
00111             blk.prevRBN = prevRBN;
00112             blk.nextRBN = -1; // set to -1 temporarily
00113
00114             stringstream blockStream;
00115             blockStream << blk.recordCount << '\n';
00116             blockStream << blk.prevRBN << '\n';
00117             blockStream << blk.nextRBN << '\n';
00118             for (const auto& rec : blk.records) blockStream << rec << '\n';
00119             string rawBlock = blockStream.str();
00120             if (rawBlock.size() > static_cast<size_t>(blockSize)) {
00121                 cerr << "Error: Block content exceeds block size." << endl;
00122                 return 1;
00123             }
00124

```

```

00125         BlockBuffer::writeBlock(out, blk, currentRBN, blockSize);
00126
00127         if (sequenceListHeadRBN == -1) {
00128             sequenceListHeadRBN = currentRBN;
00129         }
00130
00131         prevRBN = currentRBN;
00132         ++currentRBN;
00133         ++blockCount;
00134         buffer.clear();
00135         currentSize = 0;
00136     }
00137
00138     buffer.push_back(line);
00139     currentSize += lineSize;
00140
00141     if (lineCount % 5000 == 0) {
00142         cout << "[Debug] Processed " << lineCount << " lines..." << endl;
00143     }
00144 }
00145
00146 if (!buffer.empty()) {
00147     Block blk;
00148     blk.recordCount = buffer.size();
00149     blk.records = buffer;
00150     blk.prevRBN = prevRBN;
00151     blk.nextRBN = -1;
00152
00153     stringstream blockStream;
00154     blockStream << blk.recordCount << '\n';
00155     blockStream << blk.prevRBN << '\n';
00156     blockStream << blk.nextRBN << '\n';
00157     for (const auto& rec : blk.records) blockStream << rec << '\n';
00158     string rawBlock = blockStream.str();
00159     if (rawBlock.size() > static_cast<size_t>(blockSize)) {
00160         cerr << "Error: Last block content exceeds block size." << endl;
00161         return 1;
00162     }
00163
00164     BlockBuffer::writeBlock(out, blk, currentRBN, blockSize);
00165     if (sequenceListHeadRBN == -1) {
00166         sequenceListHeadRBN = currentRBN;
00167     }
00168     ++blockCount;
00169 }
00170
00171 hb.header.sequenceListHeadRBN = sequenceListHeadRBN;
00172 hb.header.blockCount = blockCount;
00173
00174 stringstream finalHeader;
00175 hb.write(finalHeader);
00176 string headerFinal = finalHeader.str();
00177
00178 if (headerFinal.size() > static_cast<size_t>(blockSize)) {
00179     cerr << "Error: Final header size exceeds block size." << endl;
00180     return 1;
00181 }
00182
00183 headerFinal.resize(blockSize, ' ');
00184 out.seekp(0);
00185 out.write(headerFinal.c_str(), blockSize);
00186
00187 cout << "[Info] Total blocks written: " << blockCount << endl;
00188 cout << "Blocked sequence set file generated: " << outputFile << endl;
00189 return 0;
00190 }

```

5.9 CSVBuffer.cpp File Reference

Implementation of [CSVBuffer](#) class functions.

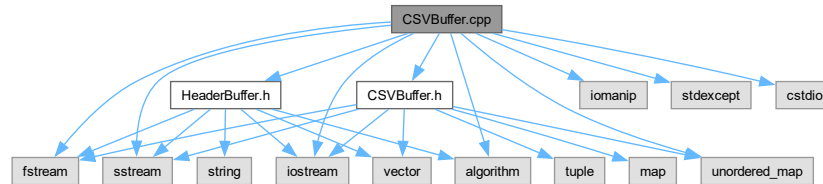
```

#include "CSVBuffer.h"
#include "HeaderBuffer.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <iomanip>

```

```
#include <stdexcept>
#include <algorithm>
#include <unordered_map>
#include <cstdio>
```

Include dependency graph for CSVBuffer.cpp:



5.9.1 Detailed Description

Implementation of [CSVBuffer](#) class functions.

This file provides functionality for reading CSV or length-indicated ZIP code data, building a primary key index, converting CSV to a structured format, and generating summaries.

Definition in file [CSVBuffer.cpp](#).

5.10 CSVBuffer.cpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #include "CSVBuffer.h"
00007 #include "HeaderBuffer.h"
00008 #include <iostream>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <iomanip>
00012 #include <stdexcept>
00013 #include <algorithm>
00014 #include <unordered_map>
00015 #include <cstdio>
00016
00017 using namespace std;
00018
00019 CSVBuffer::CSVBuffer(const string& filename) {
00020     file.open(filename);
00021     if (!file.is_open()) {
00022         throw runtime_error("Could not open file");
00023     }
00024
00025     Header header;
00026     if (header.readFromFile(file)) {
00027         cout << "Header successfully loaded from file." << endl;
00028         loadLengthIndicatedRecords();
00029     } else {
00030         loadRecords();
00031     }
00032 }
00033
00034 unordered_map<int, ZipRecord> CSVBuffer::buildPrimaryKeyIndex() const {
00035     unordered_map<int, ZipRecord> index;
00036     for (const auto& record : records) {
00037         index[record.zipCode] = record;
00038     }
00039     return index;
00040 }
```

```

00041
00042 void CSVBuffer::searchByZipCodes(const vector<int>& zipCodes) const {
00043     auto index = buildPrimaryKeyIndex();
00044     for (int zip : zipCodes) {
00045         auto it = index.find(zip);
00046         if (it != index.end()) {
00047             const auto& r = it->second;
00048             cout << "Zip Code: " << r.zipCode << ", Place Name: " << r.placeName
00049                 << ", State: " << r.state << ", County: " << r.county
00050                 << ", Latitude: " << r.latitude << ", Longitude: " << r.longitude << endl;
00051         } else {
00052             cout << "Zip Code " << zip << " not found in the file." << endl;
00053         }
00054     }
00055 }
00056
00057 void CSVBuffer::loadRecords() {
00058     string line;
00059     getline(file, line); // Skip header
00060     while (getline(file, line)) {
00061         stringstream ss(line);
00062         ZipRecord record;
00063         string temp;
00064
00065         getline(ss, temp, ',');
00066         if (temp.empty()) continue;
00067         try { record.zipCode = stoi(temp); } catch (...) { continue; }
00068         getline(ss, record.placeName, ',');
00069         getline(ss, record.state, ',');
00070         getline(ss, record.county, ',');
00071
00072         getline(ss, temp, ',');
00073         try { record.latitude = stod(temp); } catch (...) { continue; }
00074         getline(ss, temp, ',');
00075         try { record.longitude = stod(temp); } catch (...) { continue; }
00076
00077         records.push_back(record);
00078     }
00079 }
00080
00081 void CSVBuffer::loadLengthIndicatedRecords() {
00082     string line;
00083     while (getline(file, line)) {
00084         stringstream ss(line);
00085         string lenStr, data;
00086         getline(ss, lenStr, ',');
00087         getline(ss, data);
00088
00089         stringstream dataStream(data);
00090         ZipRecord record;
00091         string temp;
00092
00093         getline(dataStream, temp, ',');
00094         try { record.zipCode = stoi(temp); } catch (...) { continue; }
00095         getline(dataStream, record.placeName, ',');
00096         getline(dataStream, record.state, ',');
00097         getline(dataStream, record.county, ',');
00098
00099         getline(dataStream, temp, ',');
00100         try { record.latitude = stod(temp); } catch (...) { continue; }
00101         getline(dataStream, temp, ',');
00102         try { record.longitude = stod(temp); } catch (...) { continue; }
00103
00104         records.push_back(record);
00105     }
00106 }
00107
00108 void CSVBuffer::convertCSVToLengthIndicated(const string& inputFile, const string& outputFile) {
00109     ifstream in(inputFile);
00110     if (!in.is_open()) {
00111         cerr << "Error opening input file." << endl;
00112         return;
00113     }
00114
00115     ofstream tempOut("temp_data.tmp");
00116     if (!tempOut.is_open()) {
00117         cerr << "Error creating temporary file." << endl;
00118         return;
00119     }
00120
00121     string headerLine;
00122     getline(in, headerLine);
00123     headerLine.erase(remove(headerLine.begin(), headerLine.end(), '"'), headerLine.end());
00124     headerLine.erase(remove(headerLine.begin(), headerLine.end(), '\\n'), headerLine.end());
00125
00126     stringstream ss(headerLine);
00127     string fieldName;

```



```

00128     vector<Field> fields;
00129     int index = 0;
00130     int primaryKeyIndex = 0;
00131
00132     cout << "Parsed Fields:" << endl;
00133     while (getline(ss, fieldName, ',')) {
00134         string rawField = fieldName;
00135         fieldName.erase(remove_if(fieldName.begin(), fieldName.end(), ::isspace), fieldName.end());
00136
00137         Field field;
00138         field.name = fieldName;
00139         field.type = "string";
00140         field.format = "text/csv";
00141         field.isPrimaryKey = false;
00142         fields.push_back(field);
00143
00144         cout << "  Field " << index << ": '" << rawField << "' -> '" << fieldName << "'" << endl;
00145         ++index;
00146     }
00147
00148     if (!fields.empty()) {
00149         fields[0].isPrimaryKey = true;
00150         primaryKeyIndex = 0;
00151     }
00152
00153     int sizeInBytes = 50;
00154     for (const auto& field : fields) {
00155         sizeInBytes += field.name.length() + field.type.length() + field.format.length() + 10;
00156     }
00157
00158     int recordCount = 0;
00159     string line;
00160     while (getline(in, line)) {
00161         line.erase(remove(line.begin(), line.end(), '"'), line.end());
00162         if (line.empty()) continue;
00163
00164         int length = line.size();
00165         tempOut << setw(4) << setfill('0') << length << "," << line << endl;
00166         ++recordCount;
00167     }
00168
00169     in.close();
00170     tempOut.close();
00171
00172     ofstream out(outputFile);
00173     if (!out.is_open()) {
00174         cerr << "Error creating output file." << endl;
00175         return;
00176     }
00177
00178     Header header("LengthIndicated", "1.0", sizeInBytes, "ASCII", "zipcode.idx",
00179                 recordCount, fields, fields[primaryKeyIndex].name);
00180     header.writeToFile(out);
00181
00182     ifstream tempIn("temp_data.tmp");
00183     out << tempIn.rdbuf();
00184     tempIn.close();
00185     out.close();
00186     remove("temp_data.tmp");
00187 }
00188
00189 void CSVBuffer::generateStateTable() const {
00190     vector<ZipRecord> sortedRecords = records;
00191     string sortField;
00192     cout << "Choose sorting field (zipCode, placeName, state, latitude, longitude): ";
00193     cin >> sortField;
00194
00195     if (sortField == "zipCode") {
00196         sort(sortedRecords.begin(), sortedRecords.end(), [](const ZipRecord& a, const ZipRecord& b) {
00197             return a.zipCode < b.zipCode;
00198         });
00199     } else if (sortField == "placeName") {
00200         sort(sortedRecords.begin(), sortedRecords.end(), [](const ZipRecord& a, const ZipRecord& b) {
00201             return a.placeName < b.placeName;
00202         });
00203     } else if (sortField == "state") {
00204         sort(sortedRecords.begin(), sortedRecords.end(), [](const ZipRecord& a, const ZipRecord& b) {
00205             return a.state < b.state;
00206         });
00207     } else if (sortField == "latitude") {
00208         sort(sortedRecords.begin(), sortedRecords.end(), [](const ZipRecord& a, const ZipRecord& b) {
00209             return a.latitude < b.latitude;
00210         });
00211     } else if (sortField == "longitude") {
00212         sort(sortedRecords.begin(), sortedRecords.end(), [](const ZipRecord& a, const ZipRecord& b) {
00213             return a.longitude < b.longitude;
00214         });
00215     }
}

```

```

00215     } else {
00216         cerr << "Invalid sorting field. Defaulting to state." << endl;
00217     }
00218
00219     map<string, tuple<ZipRecord, ZipRecord, ZipRecord, ZipRecord> stateData;
00220     for (const auto& record : sortedRecords) {
00221         auto& [east, west, north, south] = stateData[record.state];
00222         if (east.zipCode == 0 || record.longitude < east.longitude) east = record;
00223         if (west.zipCode == 0 || record.longitude > west.longitude) west = record;
00224         if (north.zipCode == 0 || record.latitude > north.latitude) north = record;
00225         if (south.zipCode == 0 || record.latitude < south.latitude) south = record;
00226     }
00227
00228     string outputFilename;
00229     cout << "Enter output filename: ";
00230     cin >> outputFilename;
00231
00232     ofstream outFile(outputFilename);
00233     if (!outFile.is_open()) {
00234         cerr << "Error: Could not open output file." << endl;
00235         return;
00236     }
00237
00238     outFile << "State, Easternmost, Westernmost, Northernmost, Southernmost" << endl;
00239     for (const auto& [state, extremes] : stateData) {
00240         const auto& [east, west, north, south] = extremes;
00241         outFile << state << ", " << east.zipCode << ", " << west.zipCode << ", "
00242             << north.zipCode << ", " << south.zipCode << endl;
00243     }
00244
00245     outFile.close();
00246     cout << "Summary written to " << outputFilename << endl;
00247 }

```

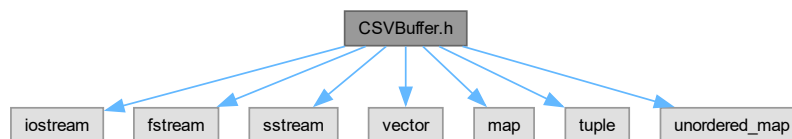
5.11 CSVBuffer.h File Reference

```

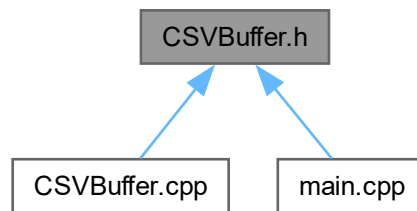
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <map>
#include <tuple>
#include <unordered_map>

```

Include dependency graph for CSVBuffer.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [ZipRecord](#)
Structure to store zip code information.
- class [CSVBuffer](#)
Class to handle CSV data and extract structured zip code records.

5.12 CSVBuffer.h

[Go to the documentation of this file.](#)

```

00001
00004
00005 #ifndef CSVBUFFER_H
00006 #define CSVBUFFER_H
00007
00008 #include <iostream>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <vector>
00012 #include <map>
00013 #include <tuple>
00014 #include <unordered_map>
00015
00016 using namespace std;
00017
00019 struct ZipRecord {
00020     int zipCode;
00021     string placeName;
00022     string state;
00023     string county;
00024     double latitude;
00025     double longitude;
00026 };
00027
00029 class CSVBuffer {
00030 private:
00031     ifstream file;
00032     vector<ZipRecord> records;
00033
00035     void loadRecords();
00036
00038     void loadLengthIndicatedRecords();
00039
00040 public:
00043     CSVBuffer(const string& filename);
00044
00047     const vector<ZipRecord>& getRecords() const;
00048
00051     void generateStateTable() const;
00052

```

```

00056     static void convertCSVToLengthIndicated(const string& inputFile, const string& outputFile);
00057
00060     unordered_map<int, ZipRecord> buildPrimaryKeyIndex() const;
00061
00064     void searchByZipCodes(const vector<int>& zipCodes) const;
00065 };
00066
00067 #endif // CSVBUFFER_H

```

5.13 DeleteRecord.cpp File Reference

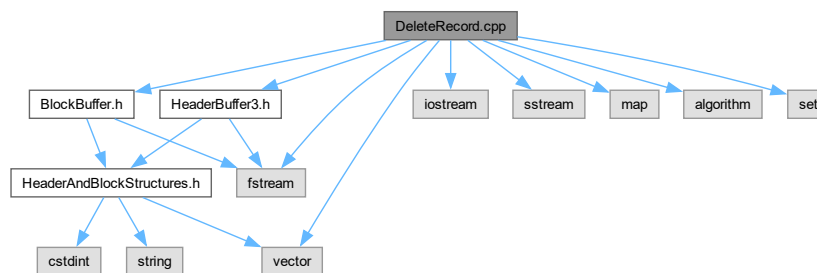
Handles deletion of ZIP code records from a blocked sequence set.

```

#include "HeaderBuffer3.h"
#include "BlockBuffer.h"
#include <fstream>
#include <iostream>
#include <sstream>
#include <vector>
#include <map>
#include <algorithm>
#include <set>

```

Include dependency graph for DeleteRecord.cpp:



Functions

- string [extractKey](#) (const string &record)
- map< string, int > [loadIndex](#) (const string &indexFile)
- void [writeIndex](#) (const string &indexFile, const map< string, int > &index)
- int [main](#) (int argc, char *argv[])

5.13.1 Detailed Description

Handles deletion of ZIP code records from a blocked sequence set.

Definition in file [DeleteRecord.cpp](#).

5.13.2 Function Documentation

5.13.2.1 extractKey()

```
string extractKey (  
    const string & record)
```

Definition at line 16 of file [DeleteRecord.cpp](#).

Here is the caller graph for this function:



5.13.2.2 loadIndex()

```
map< string, int > loadIndex (  
    const string & indexFile)
```

Definition at line 23 of file [DeleteRecord.cpp](#).

Here is the caller graph for this function:

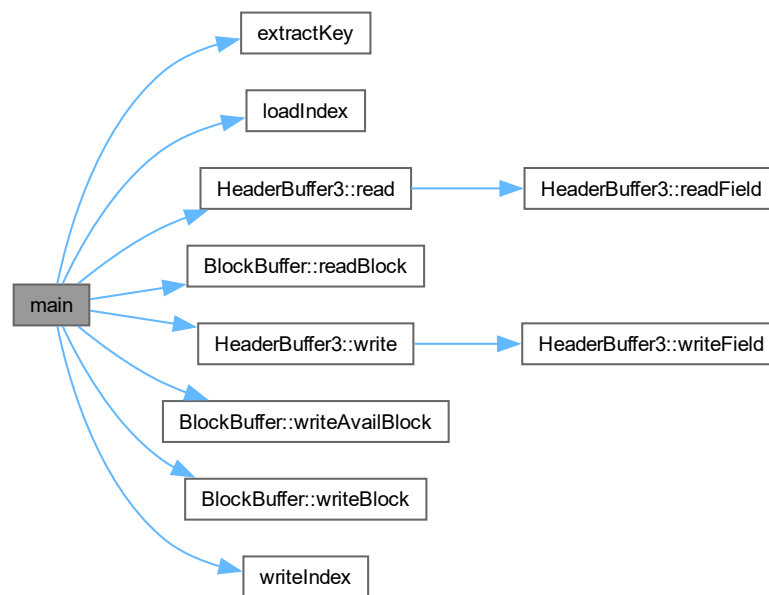


5.13.2.3 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 44 of file [DeleteRecord.cpp](#).

Here is the call graph for this function:



5.13.2.4 writeIndex()

```

void writeIndex (
    const string & indexFile,
    const map< string, int > & index)
  
```

Definition at line 37 of file [DeleteRecord.cpp](#).

Here is the caller graph for this function:



5.14 DeleteRecord.cpp

[Go to the documentation of this file.](#)

```

00001
00003
00004 #include "HeaderBuffer3.h"
  
```

```

00005 #include "BlockBuffer.h"
00006 #include <fstream>
00007 #include <iostream>
00008 #include <sstream>
00009 #include <vector>
00010 #include <map>
00011 #include <algorithm>
00012 #include <set>
00013
00014 using namespace std;
00015
00016 string extractKey(const string& record) {
00017     stringstream ss(record);
00018     string key;
00019     getline(ss, key, ',');
00020     return key;
00021 }
00022
00023 map<string, int> loadIndex(const string& indexFile) {
00024     map<string, int> index;
00025     ifstream in(indexFile);
00026     string line;
00027     while (getline(in, line)) {
00028         stringstream ss(line);
00029         string key; int rbn;
00030         getline(ss, key, ',');
00031         ss » rbn;
00032         index[key] = rbn;
00033     }
00034     return index;
00035 }
00036
00037 void writeIndex(const string& indexFile, const map<string, int>& index) {
00038     ofstream out(indexFile);
00039     for (const auto& [key, rbn] : index) {
00040         out « key « "," « rbn « endl;
00041     }
00042 }
00043
00044 int main(int argc, char* argv[]) {
00045     if (argc < 4) {
00046         cerr « "Usage: " « argv[0] « " <blocked_file> <index_file> <delete_keys_file>" « endl;
00047         return 1;
00048     }
00049
00050     string blockedFile = argv[1];
00051     string indexFile = argv[2];
00052     string deleteFile = argv[3];
00053
00054     map<string, int> index = loadIndex(indexFile);
00055     if (index.empty()) {
00056         cerr « "Empty or missing index file." « endl;
00057         return 1;
00058     }
00059
00060     fstream data(blockedFile, ios::in | ios::out | ios::binary);
00061     if (!data.is_open()) {
00062         cerr « "Failed to open blocked file." « endl;
00063         return 1;
00064     }
00065
00066     HeaderBuffer3 hb;
00067     if (!hb.read(data)) {
00068         cerr « "Failed to read header." « endl;
00069         return 1;
00070     }
00071
00072     int blockSize = hb.header.blockSize;
00073     set<string> deleted;
00074
00075     ifstream delFile(deleteFile);
00076     string zip;
00077     while (getline(delFile, zip)) {
00078         auto it = index.lower_bound(zip);
00079         if (it == index.end()) {
00080             cout « "Zip Code " « zip « " not found in index." « endl;
00081             continue;
00082         }
00083
00084         int rbn = it->second;
00085         Block blk;
00086         if (!BlockBuffer::readBlock(data, blk, rbn, blockSize)) continue;
00087
00088         blk.records.erase(remove_if(blk.records.begin(), blk.records.end(), [&](const string& rec) {
00089             return extractKey(rec) == zip;
00090         }), blk.records.end());
00091

```

```

00092         if (blk.records.empty()) {
00093             blk.recordCount = 0;
00094             {
00095                 fstream outFile(blockedFile, ios::in | ios::out | ios::binary);
00096                 outFile.seekp(data.tellp());
00097                 BlockBuffer::writeAvailBlock(outFile, AvailBlock(blockSize), rbn, blockSize);
00098             }
00099             index.erase(it);
00100             cout << "Deleted entire block RBN " << rbn << endl;
00101         } else {
00102             blk.recordCount = blk.records.size();
00103             {
00104                 fstream outFile(blockedFile, ios::in | ios::out | ios::binary);
00105                 outFile.seekp(data.tellp());
00106                 BlockBuffer::writeBlock(outFile, blk, rbn, blockSize);
00107             }
00108
00109             string newMaxKey = extractKey(blk.records.back());
00110             if (extractKey(it->first) != newMaxKey) {
00111                 index.erase(it);
00112                 index[newMaxKey] = rbn;
00113             }
00114             cout << "Deleted zip " << zip << " from RBN " << rbn << endl;
00115         }
00116
00117         deleted.insert(zip);
00118         --hb.header.recordCount;
00119     }
00120
00121     data.close();
00122     ofstream outFile(blockedFile, ios::out | ios::binary);
00123     if (!outFile.is_open()) {
00124         cerr << "Failed to reopen blocked file for writing header." << endl;
00125         return 1;
00126     }
00127     hb.write(outFile);
00128     writeIndex(indexFile, index);
00129
00130     cout << "Deleted " << deleted.size() << " records." << endl;
00131     return 0;
00132 }

```

5.15 DumpBlocked.cpp File Reference

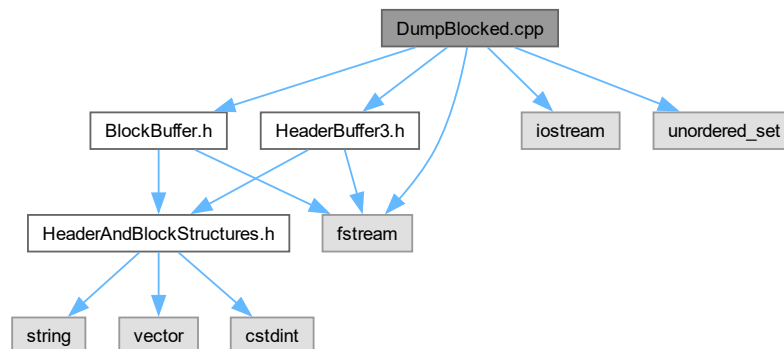
Utility to dump blocked sequence set content in physical or logical order.

```

#include "HeaderBuffer3.h"
#include "BlockBuffer.h"
#include <fstream>
#include <iostream>
#include <unordered_set>

```

Include dependency graph for DumpBlocked.cpp:



Functions

- void [dumpPhysical](#) (const string &filename, int blockSize, int blockCount)
Dumps all blocks in physical order.
- void [dumpLogical](#) (const string &filename, int blockSize, int startRBN)
Dumps blocks in logical sequence order based on header links.
- int [main](#) (int argc, char *argv[])

5.15.1 Detailed Description

Utility to dump blocked sequence set content in physical or logical order.

Definition in file [DumpBlocked.cpp](#).

5.15.2 Function Documentation

5.15.2.1 dumpLogical()

```
void dumpLogical (  
    const string & filename,  
    int blockSize,  
    int startRBN)
```

Dumps blocks in logical sequence order based on header links.

Definition at line 40 of file [DumpBlocked.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.15.2.2 dumpPhysical()

```
void dumpPhysical (  
    const string & filename,  
    int blockSize,  
    int blockCount)
```

Dumps all blocks in physical order.

Definition at line 13 of file [DumpBlocked.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:

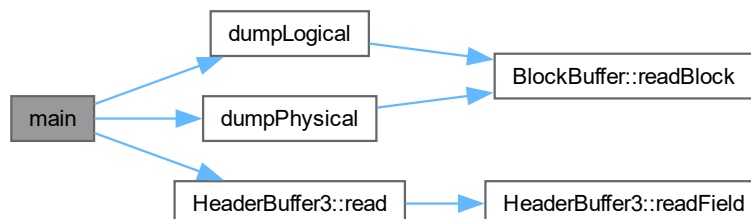


5.15.2.3 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 71 of file [DumpBlocked.cpp](#).

Here is the call graph for this function:



5.16 DumpBlocked.cpp

[Go to the documentation of this file.](#)

```

00001
00002
00003
00004 #include "HeaderBuffer3.h"
00005 #include "BlockBuffer.h"
00006 #include <fstream>
00007 #include <iostream>
00008 #include <unordered_set>
00009
00010 using namespace std;
00011
00012 void dumpPhysical(const string& filename, int blockSize, int blockCount) {
00013     ifstream in(filename, ios::binary);
00014     if (!in.is_open()) {
00015         cerr << "Error opening file for physical dump." << endl;
00016         return;
00017     }
00018
00019     cout << "\n--- Physical Dump ---\n";
00020     for (int rbn = 0; rbn < blockCount; ++rbn) {
00021         Block blk;
00022         BlockBuffer::readBlock(in, blk, rbn, blockSize);
00023
00024         if (blk.recordCount == 0) {
00025             cout << "RBN " << rbn << ": *available*" << endl;
00026         } else {
00027             cout << "RBN " << rbn << ": ";
00028             for (const auto& rec : blk.records) {
00029                 string key = rec.substr(5, 5); // assume ZIP is field 0, 5 digits
00030                 cout << key << " ";
00031             }
00032             cout << "→ RBN " << blk.nextRBN << endl;
00033         }
00034     }
00035     in.close();
00036 }
00037
00038 void dumpLogical(const string& filename, int blockSize, int startRBN) {
00039     ifstream in(filename, ios::binary);
00040     if (!in.is_open()) {
00041         cerr << "Error opening file for logical dump." << endl;
00042         return;
00043     }
00044
00045     cout << "\n--- Logical Dump ---\n";
00046     int current = startRBN;
00047     unordered_set<int> visited;
00048
00049     while (current != -1 && visited.find(current) == visited.end()) {
00050         visited.insert(current);
00051         Block blk;
00052         BlockBuffer::readBlock(in, blk, current, blockSize);
00053
00054         if (blk.recordCount == 0) {
00055             cout << "RBN " << current << ": *available*" << endl;
00056         } else {
00057             cout << "RBN " << current << ": ";
00058             for (const auto& rec : blk.records) {
00059                 string key = rec.substr(5, 5); // assume ZIP is field 0
00060                 cout << key << " ";
00061             }
00062             cout << "→ RBN " << blk.nextRBN << endl;
00063         }
00064         current = blk.nextRBN;
00065     }
00066     in.close();
00067 }
00068
00069 int main(int argc, char* argv[]) {
00070     if (argc < 3) {
00071         cerr << "Usage: " << argv[0] << " <blocked_file> <physical|logical>" << endl;
00072         return 1;
00073     }
00074
00075     string filename = argv[1];
00076     string mode = argv[2];
00077
00078     ifstream in(filename, ios::binary);
00079     HeaderBuffer3 hb;
00080     if (!hb.read(in)) {
00081         cerr << "Failed to read header." << endl;
00082         return 1;
00083     }
00084 }

```

```

00086     in.close();
00087
00088     if (mode == "physical") {
00089         dumpPhysical(filename, hb.header.blockSize, hb.header.blockCount);
00090     } else if (mode == "logical") {
00091         dumpLogical(filename, hb.header.blockSize, hb.header.sequenceListHeadRBN);
00092     } else {
00093         cerr << "Invalid mode. Use 'physical' or 'logical'." << endl;
00094         return 1;
00095     }
00096
00097     return 0;
00098 }

```

5.17 DumpUtility.cpp File Reference

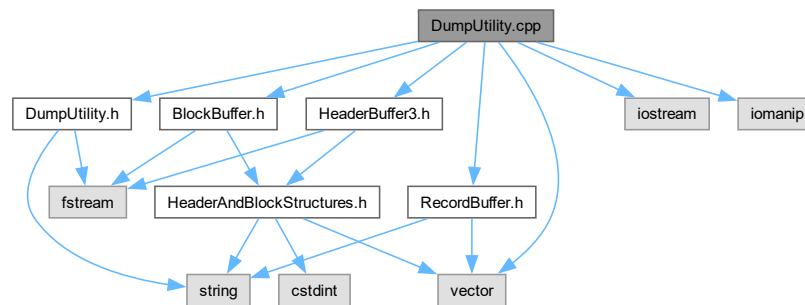
Implements dump functions for the blocked sequence set structure.

```

#include "DumpUtility.h"
#include "HeaderBuffer3.h"
#include "BlockBuffer.h"
#include "RecordBuffer.h"
#include <iostream>
#include <vector>
#include <iomanip>

```

Include dependency graph for DumpUtility.cpp:



Functions

- void [printBlock](#) (int rbn, const [Block](#) &block)
Prints a block's RBN, keys, and next RBN.
- void [printAvail](#) (int rbn, const [AvailBlock](#) &avail)
Prints an avail block's RBN and its successor.
- void [dumpByPhysicalOrder](#) (const string &filename)
Dumps all blocks in physical order from the file.
- void [dumpByLogicalOrder](#) (const string &filename)
Dumps all blocks in logical order using nextRBN links.

5.17.1 Detailed Description

Implements dump functions for the blocked sequence set structure.

Definition in file [DumpUtility.cpp](#).

5.17.2 Function Documentation

5.17.2.1 dumpByLogicalOrder()

```
void dumpByLogicalOrder (  
    const string & filename)
```

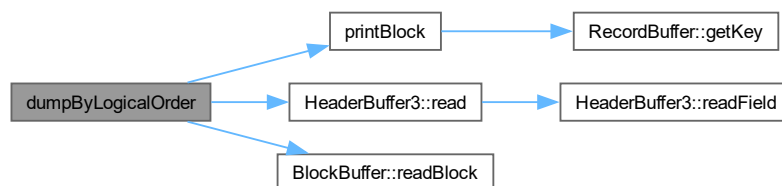
Dumps all blocks in logical order using nextRBN links.

Parameters

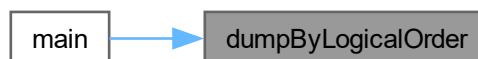
<i>filename</i>	The blocked sequence set file.
-----------------	--------------------------------

Definition at line 82 of file [DumpUtility.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.17.2.2 dumpByPhysicalOrder()

```
void dumpByPhysicalOrder (  
    const string & filename)
```

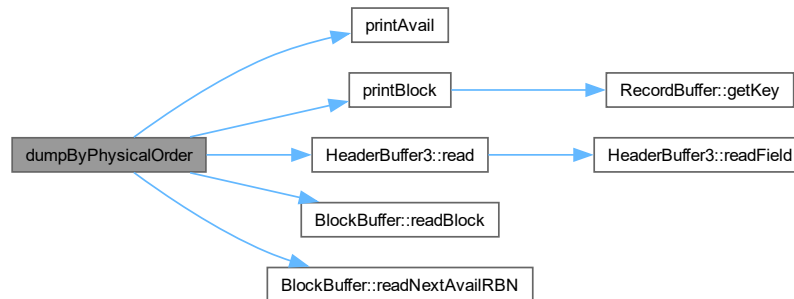
Dumps all blocks in physical order from the file.

Parameters

<i>filename</i>	The blocked sequence set file.
-----------------	--------------------------------

Definition at line 41 of file [DumpUtility.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.17.2.3 printAvail()

```

void printAvail (
    int rbn,
    const AvailBlock & avail)
  
```

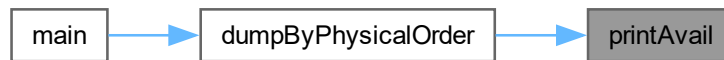
Prints an avail block's RBN and its successor.

Parameters

<i>rbn</i>	Relative Block Number of the avail block.
<i>avail</i>	The AvailBlock to print.

Definition at line 33 of file [DumpUtility.cpp](#).

Here is the caller graph for this function:



5.17.2.4 printBlock()

```
void printBlock (  
    int rbn,  
    const Block & block)
```

Prints a block's RBN, keys, and next RBN.

Parameters

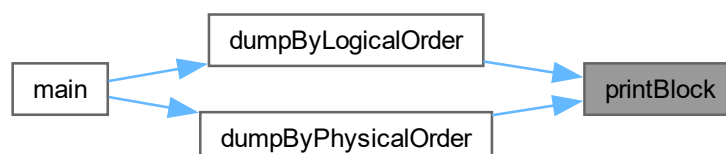
<i>rbn</i>	Relative Block Number.
<i>block</i>	The Block structure to print.

Definition at line 19 of file [DumpUtility.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.18 DumpUtility.cpp

[Go to the documentation of this file.](#)

```

00001
00002
00003
00004 #include "DumpUtility.h"
00005 #include "HeaderBuffer3.h"
00006 #include "BlockBuffer.h"
00007 #include "RecordBuffer.h"
00008 #include <iostream>
00009 #include <vector>
00010 #include <iomanip>
00011
00012 using namespace std;
00013
00014 void printBlock(int rbn, const Block& block) {
00015     cout << rbn << " ";
00016     for (const string& rec : block.records) {
00017         RecordBuffer rb(rec); // Extract fields using record buffer
00018         cout << rb.getKey() << " "; // Print key field
00019     }
00020     cout << " " << block.nextRBN << endl; // Show next RBN link
00021 }
00022
00023 void printAvail(int rbn, const AvailBlock& avail) {
00024     cout << rbn << " *available* " << avail.nextAvailRBN << endl;
00025 }
00026
00027 void dumpByPhysicalOrder(const string& filename) {
00028     ifstream in(filename, ios::in | ios::binary);
00029     if (!in.is_open()) {
00030         cerr << "Could not open file: " << filename << endl;
00031         return;
00032     }
00033
00034     HeaderBuffer3 hb;
00035     if (!hb.read(in)) {
00036         cerr << "Failed to read header." << endl;
00037         return;
00038     }
00039
00040     int blockSize = hb.header.blockSize;
00041
00042     // Print header pointers
00043     cout << "List Head: " << hb.header.sequenceListHeadRBN << endl;
00044     cout << "Avail Head: " << hb.header.availListHeadRBN << endl;
00045
00046     // Traverse blocks sequentially by physical RBN
00047     for (int rbn = 0; rbn < hb.header.blockCount; ++rbn) {
00048         in.clear(); // Clear EOF/failure flags
00049         in.seekg(0); // Reset file position
00050
00051         Block blk;
00052         if (BlockBuffer::readBlock(in, blk, rbn, blockSize)) {
00053             if (blk.recordCount == 0) {
00054                 // Handle avail block
00055                 AvailBlock avail(BlockBuffer::readNextAvailRBN(in, rbn, blockSize));
00056                 printAvail(rbn, avail);
00057             } else {
00058                 printBlock(rbn, blk);
00059             }
00060         }
00061     }
00062 }
00063
00064 void dumpByLogicalOrder(const string& filename) {
00065     ifstream in(filename, ios::in | ios::binary);
00066     if (!in.is_open()) {
00067         cerr << "Could not open file: " << filename << endl;
00068         return;
00069     }
00070
00071     HeaderBuffer3 hb;
00072     if (!hb.read(in)) {
00073         cerr << "Failed to read header." << endl;
00074         return;
00075     }
00076
00077     int blockSize = hb.header.blockSize;
00078
00079     // Print header pointers
00080     cout << "List Head: " << hb.header.sequenceListHeadRBN << endl;
00081     cout << "Avail Head: " << hb.header.availListHeadRBN << endl;
00082
00083     // Follow logical linked list using nextRBN

```



```

00102     int rbn = hb.header.sequenceListHeadRBN;
00103     while (rbn != -1) {
00104         Block blk;
00105         if (BlockBuffer::readBlock(in, blk, rbn, blockSize)) {
00106             printBlock(rbn, blk);
00107             rbn = blk.nextRBN; // Move to next logical block
00108         } else {
00109             break; // Stop on read failure
00110         }
00111     }
00112 }

```

5.19 DumpUtility.h File Reference

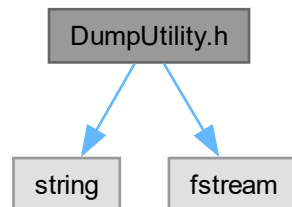
Provides functions to dump the blocked sequence set file structure.

```

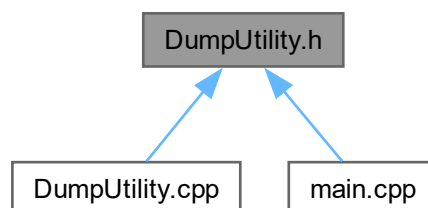
#include <string>
#include <fstream>

```

Include dependency graph for DumpUtility.h:



This graph shows which files directly or indirectly include this file:



Functions

- void [dumpByPhysicalOrder](#) (const std::string &filename)
Dumps blocks in physical order (RBN 0 to N) to stdout.
- void [dumpByLogicalOrder](#) (const std::string &filename)
Dumps blocks in logical order (follows linked list of active blocks).

5.19.1 Detailed Description

Provides functions to dump the blocked sequence set file structure.

Definition in file [DumpUtility.h](#).

5.19.2 Function Documentation

5.19.2.1 dumpByLogicalOrder()

```
void dumpByLogicalOrder (
    const std::string & filename)
```

Dumps blocks in logical order (follows linked list of active blocks).

Parameters

<i>filename</i>	Name of the blocked sequence set file to dump.
-----------------	--

5.19.2.2 dumpByPhysicalOrder()

```
void dumpByPhysicalOrder (
    const std::string & filename)
```

Dumps blocks in physical order (RBN 0 to N) to stdout.

Parameters

<i>filename</i>	Name of the blocked sequence set file to dump.
-----------------	--

5.20 DumpUtility.h

[Go to the documentation of this file.](#)

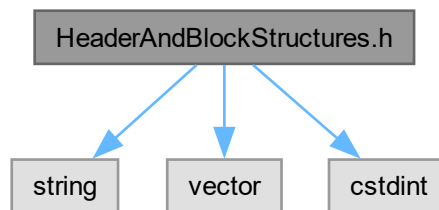
```
00001
00003
00004 #ifndef DUMPUTILITY_H
00005 #define DUMPUTILITY_H
00006
00007 #include <string>
00008 #include <fstream>
00009
00012 void dumpByPhysicalOrder(const std::string& filename);
00013
00016 void dumpByLogicalOrder(const std::string& filename);
00017
00018 #endif // DUMPUTILITY_H
```

5.21 HeaderAndBlockStructures.h File Reference

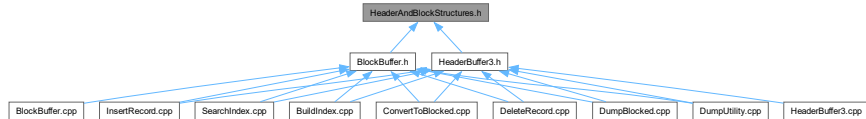
Defines structures for the header and block layouts used in the Blocked Sequence Set.

```
#include <string>
#include <vector>
#include <stdint>
```

Include dependency graph for HeaderAndBlockStructures.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [Field](#)
Structure representing metadata for a single field in the record.
- class [Header](#)
Structure representing the header of a blocked sequence set file.
- struct [Block](#)
Structure for an active block in the blocked sequence set.
- struct [AvailBlock](#)
Structure for a free/available block.

5.21.1 Detailed Description

Defines structures for the header and block layouts used in the Blocked Sequence Set.

Definition in file [HeaderAndBlockStructures.h](#).

5.22 HeaderAndBlockStructures.h

[Go to the documentation of this file.](#)

```

00001
00002
00003
00004 #ifndef HEADER_AND_BLOCK_STRUCTURES_H
00005 #define HEADER_AND_BLOCK_STRUCTURES_H
00006
00007 #include <string>
00008 #include <vector>
00009 #include <cstdint>
00010
00011 using namespace std;
00012
00013 struct Field {
00014     string name;
00015     string type;
00016     string format;
00017     bool isPrimaryKey;
00018 };
00019
00020 struct Header {
00021     string fileStructureType;
00022     string version;
00023     int headerSize;
00024     int recordSizeFieldLength;
00025     string sizeFormatType;
00026     int blockSize;
00027     float minBlockCapacity;
00028     string indexFileName;
00029     string indexSchema;
00030     int recordCount;
00031     int blockCount;
00032     int fieldsPerRecord;
00033     vector<Field> fields;
00034     int primaryKeyOrdinal;
00035     int availListHeadRBN;
00036     int sequenceListHeadRBN;
00037     bool stale;
00038 };
00039
00040 struct Block {
00041     int recordCount;
00042     int prevRBN;
00043     int nextRBN;
00044     vector<string> records;
00045 };
00046
00047 struct AvailBlock {
00048     int recordCount = 0;
00049     int nextAvailRBN;
00050     string blankFill;
00051
00052     AvailBlock(int blockSize) {
00053         blankFill = string(blockSize - sizeof(int) * 2, ' ');
00054     }
00055 };
00056
00057 #endif // HEADER_AND_BLOCK_STRUCTURES_H

```

5.23 HeaderBuffer.h File Reference

Defines a class for handling header records in data files.

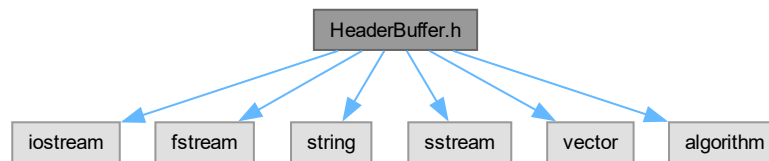
```

#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
#include <vector>

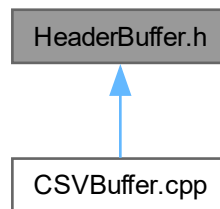
```

```
#include <algorithm>
```

Include dependency graph for HeaderBuffer.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [Field](#)
Structure representing metadata for a single field in the record.
- class [Header](#)
Structure representing the header of a blocked sequence set file.

5.23.1 Detailed Description

Defines a class for handling header records in data files.

This class provides functionality to read and write the header record of a length-indicated CSV file, supporting structured metadata.

Definition in file [HeaderBuffer.h](#).

5.24 HeaderBuffer.h

[Go to the documentation of this file.](#)

```

00001
00005
00006 #ifndef HEADERBUFFER_H
00007 #define HEADERBUFFER_H
00008
00009 #include <iostream>
00010 #include <fstream>
00011 #include <string>
00012 #include <sstream>
00013 #include <vector>
00014 #include <algorithm>
00015
00016 using namespace std;
00017
00019 struct Field {
00020     string name;
00021     string type;
00022     string format;
00023     bool isPrimaryKey = false;
00024 };
00025
00027 class Header {
00028 public:
00029     string fileStructureType;
00030     string version;
00031     int sizeInBytes;
00032     string sizeFormatType;
00033     string primaryKeyFileName;
00034     int recordCount;
00035     int fieldsPerRecord;
00036     vector<Field> fields;
00037     string primaryKey;
00038
00040     Header()
00041         : fileStructureType(""), version(""), sizeInBytes(0), sizeFormatType("ASCII"),
00042           primaryKeyFileName(""), recordCount(0), fieldsPerRecord(0), primaryKey("") {}
00043
00045     Header(const string& fileStructureType, const string& version, int sizeInBytes,
00046           const string& sizeFormatType, const string& primaryKeyFileName,
00047           int recordCount, const vector<Field>& fields, const string& primaryKey)
00048         : fileStructureType(fileStructureType), version(version), sizeInBytes(sizeInBytes),
00049           sizeFormatType(sizeFormatType), primaryKeyFileName(primaryKeyFileName),
00050           recordCount(recordCount), fieldsPerRecord(fields.size()),
00051           fields(fields), primaryKey(primaryKey) {
00052         // Sanitize primary key
00053         this->primaryKey.erase(remove_if(this->primaryKey.begin(), this->primaryKey.end(),
::isspace), this->primaryKey.end());
00054     }
00055
00058     void writeToFile(ofstream& out) const {
00059         if (!out.is_open()) {
00060             cerr << "Error: Output file is not open." << endl;
00061             return;
00062         }
00063
00064         out << "HEADER,";
00065         out << fileStructureType << "," << version << "," << sizeInBytes << "," << sizeFormatType << ","
00066             << primaryKeyFileName << "," << recordCount << "," << fieldsPerRecord << "," << primaryKey <<
"\n";
00067
00068         for (const auto& field : fields) {
00069             out << field.name << "," << field.type << "," << field.format << ","
00070                 << (field.isPrimaryKey ? "true" : "false") << "\n";
00071         }
00072     }
00073
00077     bool readFromFile(ifstream& in) {
00078         if (!in.is_open()) {
00079             cerr << "Error: Input file is not open." << endl;
00080             return false;
00081         }
00082
00083         string line;
00084         if (!getline(in, line)) return false;
00085
00086         if (line.rfind("HEADER,", 0) != 0) {
00087             cerr << "Error: Missing HEADER line." << endl;
00088             return false;
00089         }
00090
00091         stringstream ss(line.substr(7)); // skip "HEADER,"
00092         string temp;

```

```

00093
00094     getline(ss, fileStructureType, ',');
00095     getline(ss, version, ',');
00096     getline(ss, temp, ','); sizeInBytes = stoi(temp);
00097     getline(ss, sizeFormatType, ',');
00098     getline(ss, primaryKeyFileName, ',');
00099     getline(ss, temp, ','); recordCount = stoi(temp);
00100     getline(ss, temp, ','); fieldsPerRecord = stoi(temp);
00101     getline(ss, primaryKey, ',');
00102     primaryKey.erase(remove_if(primaryKey.begin(), primaryKey.end(), ::isspace),
primaryKeyKey.end());
00103
00104     fields.clear();
00105     for (int i = 0; i < fieldsPerRecord; ++i) {
00106         if (!getline(in, line)) break;
00107         stringstream fss(line);
00108         Field f;
00109         string isPK;
00110         getline(fss, f.name, ',');
00111         f.name.erase(remove_if(f.name.begin(), f.name.end(), ::isspace), f.name.end());
00112         getline(fss, f.type, ',');
00113         getline(fss, f.format, ',');
00114         getline(fss, isPK, ',');
00115         f.isPrimaryKey = (isPK == "true");
00116         fields.push_back(f);
00117     }
00118
00119     // Print full metadata
00120     cout << "Header loaded:\n";
00121     cout << "  Structure Type      : " << fileStructureType << "\n";
00122     cout << "  Version              : " << version << "\n";
00123     cout << "  Size in Bytes        : " << sizeInBytes << "\n";
00124     cout << "  Size Format Type     : " << sizeFormatType << "\n";
00125     cout << "  Primary Key File    : " << primaryKeyFileName << "\n";
00126     cout << "  Record Count        : " << recordCount << "\n";
00127     cout << "  Fields per Record   : " << fieldsPerRecord << "\n";
00128     cout << "  Primary Key         : " << primaryKey << "\n";
00129     cout << "  Fields:\n";
00130
00131     for (const auto& field : fields) {
00132         cout << "    - Name: " << field.name
00133             << ", Type: " << field.type
00134             << ", Format: " << field.format
00135             << ", Primary Key: " << (field.isPrimaryKey ? "Yes" : "No") << "\n";
00136     }
00137
00138     return true;
00139 }
00140 };
00141
00142 #endif // HEADERBUFFER_H

```

5.25 HeaderBuffer3.cpp File Reference

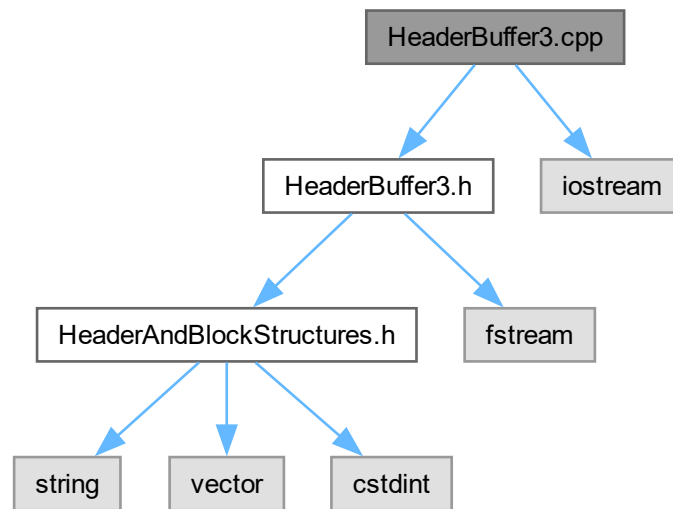
Implements methods for reading and writing blocked sequence set file headers.

```

#include "HeaderBuffer3.h"
#include <iostream>

```

Include dependency graph for HeaderBuffer3.cpp:



5.25.1 Detailed Description

Implements methods for reading and writing blocked sequence set file headers.

Definition in file [HeaderBuffer3.cpp](#).

5.26 HeaderBuffer3.cpp

[Go to the documentation of this file.](#)

```

00001
00003
00004 #include "HeaderBuffer3.h"
00005 #include <iostream>
00006
00007 using namespace std;
00008
00014 bool HeaderBuffer3::read(istream& in) {
00015     if (!in) return false;
00016
00017     // Read fixed metadata fields line-by-line
00018     getline(in, header.fileStructureType);
00019     getline(in, header.version);
00020     in » header.headerSize;
00021     in » header.recordSizeFieldLength;
00022     in.ignore(); // skip newline
00023     getline(in, header.sizeFormatType);
00024     in » header.blockSize;
00025     in » header.minBlockCapacity;
00026     in.ignore();
00027     getline(in, header.indexFileName);
00028     getline(in, header.indexSchema);
00029     in » header.recordCount;
00030     in » header.blockCount;
00031     in » header.fieldsPerRecord;
00032     in.ignore();
00033

```



```

00034     // Read field definitions
00035     header.fields.clear();
00036     for (int i = 0; i < header.fieldsPerRecord; ++i) {
00037         Field f = readField(in);
00038         header.fields.push_back(f);
00039     }
00040
00041     // Read final metadata for primary key and list heads
00042     in » header.primaryKeyOrdinal;
00043     in » header.availListHeadRBN;
00044     in » header.sequenceListHeadRBN;
00045     in » header.stale;
00046     in.ignore();
00047
00048     return true;
00049 }
00050
00055 void HeaderBuffer3::write(ostream& out) const {
00056     if (!out) return;
00057
00058     // Write basic metadata fields
00059     out « header.fileStructureType « '\n';
00060     out « header.version « '\n';
00061     out « header.headerSize « '\n';
00062     out « header.recordSizeFieldLength « '\n';
00063     out « header.sizeFormatType « '\n';
00064     out « header.blockSize « '\n';
00065     out « header.minBlockCapacity « '\n';
00066     out « header.indexFileName « '\n';
00067     out « header.indexSchema « '\n';
00068     out « header.recordCount « '\n';
00069     out « header.blockCount « '\n';
00070     out « header.fieldsPerRecord « '\n';
00071
00072     // Write field definitions
00073     for (const auto& field : header.fields) {
00074         writeField(out, field);
00075     }
00076
00077     // Write list head metadata
00078     out « header.primaryKeyOrdinal « '\n';
00079     out « header.availListHeadRBN « '\n';
00080     out « header.sequenceListHeadRBN « '\n';
00081     out « header.stale « '\n';
00082 }
00083
00089 Field HeaderBuffer3::readField(istream& in) {
00090     Field field;
00091     getline(in, field.name);
00092     getline(in, field.type);
00093     getline(in, field.format);
00094     in » field.isPrimaryKey;
00095     in.ignore();
00096     return field;
00097 }
00098
00104 void HeaderBuffer3::writeField(ostream& out, const Field& field) const {
00105     out « field.name « '\n';
00106     out « field.type « '\n';
00107     out « field.format « '\n';
00108     out « field.isPrimaryKey « '\n';
00109 }

```

5.27 HeaderBuffer3.h File Reference

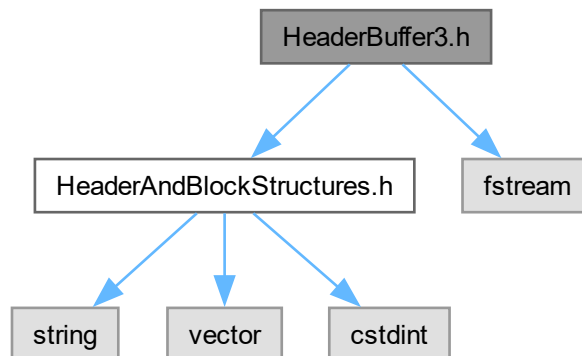
Provides a class for reading and writing header records for blocked sequence set files.

```

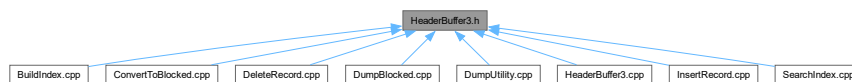
#include "HeaderAndBlockStructures.h"
#include <fstream>

```

Include dependency graph for HeaderBuffer3.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [HeaderBuffer3](#)
Class responsible for reading/writing the [Header](#) to/from a file.

5.27.1 Detailed Description

Provides a class for reading and writing header records for blocked sequence set files.

Definition in file [HeaderBuffer3.h](#).

5.28 HeaderBuffer3.h

[Go to the documentation of this file.](#)

```

00001
00002
00003
00004 #ifndef HEADERBUFFER3_H
00005 #define HEADERBUFFER3_H
00006
00007 #include "HeaderAndBlockStructures.h"
00008 #include <fstream>
00009
00010 using namespace std;
00011

```

```

00013 class HeaderBuffer3 {
00014 public:
00015     Header header;
00016
00020     bool read(istream& in);
00021
00025     void write(ostream& out) const;
00026
00027 private:
00029     Field readField(istream& in);
00030
00032     void writeField(ostream& out, const Field& field) const;
00033 };
00034
00035 #endif // HEADERBUFFER3_H

```

5.29 InsertRecord.cpp File Reference

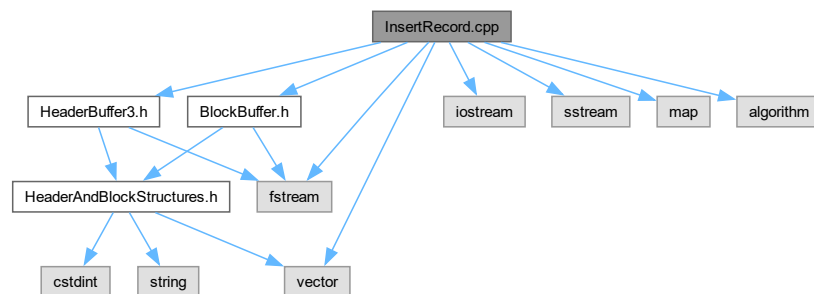
Handles insertion of new ZIP code records into a blocked sequence set.

```

#include "HeaderBuffer3.h"
#include "BlockBuffer.h"
#include <fstream>
#include <iostream>
#include <sstream>
#include <vector>
#include <map>
#include <algorithm>

```

Include dependency graph for InsertRecord.cpp:



Functions

- string [extractKey](#) (const string &record)
Extracts the primary key (ZIP code) from a record string.
- map< string, int > [loadIndex](#) (const string &indexFile)
Loads the index file into a map of <key, RBN> pairs.
- void [writeIndex](#) (const string &indexFile, const map< string, int > &index)
Writes the index map to the index file.
- int [main](#) (int argc, char *argv[])
Main function for inserting ZIP code records into a blocked sequence set.

5.29.1 Detailed Description

Handles insertion of new ZIP code records into a blocked sequence set.

Definition in file [InsertRecord.cpp](#).

5.29.2 Function Documentation

5.29.2.1 `extractKey()`

```
string extractKey (  
    const string & record)
```

Extracts the primary key (ZIP code) from a record string.

Parameters

<i>record</i>	A comma-separated string representing a ZIP code record.
---------------	--

Returns

The primary key (ZIP code).

Definition at line 18 of file [InsertRecord.cpp](#).

Here is the caller graph for this function:



5.29.2.2 `loadIndex()`

```
map< string, int > loadIndex (  
    const string & indexFile)
```

Loads the index file into a map of <key, RBN> pairs.

Parameters

<i>indexFile</i>	Path to the index file.
------------------	-------------------------

Returns

Map containing highest keys and their corresponding RBNs.

Definition at line 28 of file [InsertRecord.cpp](#).

Here is the caller graph for this function:

**5.29.2.3 main()**

```
int main (  
    int argc,  
    char * argv[])
```

Main function for inserting ZIP code records into a blocked sequence set.

Parameters

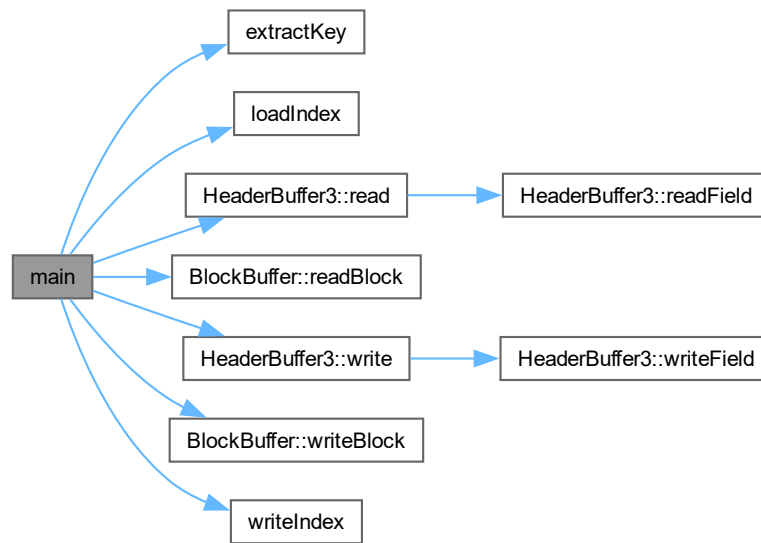
<i>argc</i>	Argument count.
<i>argv</i>	Command-line arguments: <blocked_file> <index_file> <insert_file>.

Returns

0 on success, 1 on failure.

Definition at line 56 of file [InsertRecord.cpp](#).

Here is the call graph for this function:



5.29.2.4 writeIndex()

```

void writeIndex (
    const string & indexFile,
    const map< string, int > & index)
  
```

Writes the index map to the index file.

Parameters

<i>indexFile</i>	Path to the index file.
<i>index</i>	Map containing updated <key, RBN> pairs.

Definition at line 45 of file [InsertRecord.cpp](#).

Here is the caller graph for this function:



5.30 InsertRecord.cpp

[Go to the documentation of this file.](#)

```

00001
00002
00003
00004 #include "HeaderBuffer3.h"
00005 #include "BlockBuffer.h"
00006 #include <fstream>
00007 #include <iostream>
00008 #include <sstream>
00009 #include <vector>
00010 #include <map>
00011 #include <algorithm>
00012
00013 using namespace std;
00014
00015 string extractKey(const string& record) {
00016     stringstream ss(record);
00017     string key;
00018     getline(ss, key, ',');
00019     return key;
00020 }
00021
00022 map<string, int> loadIndex(const string& indexFile) {
00023     map<string, int> index;
00024     ifstream in(indexFile);
00025     string line;
00026     while (getline(in, line)) {
00027         stringstream ss(line);
00028         string key; int rbn;
00029         getline(ss, key, ',');
00030         ss >> rbn;
00031         index[key] = rbn;
00032     }
00033     return index;
00034 }
00035
00036 void writeIndex(const string& indexFile, const map<string, int>& index) {
00037     ofstream out(indexFile);
00038     for (const auto& [key, rbn] : index) {
00039         out << key << "," << rbn << endl;
00040     }
00041 }
00042
00043 int main(int argc, char* argv[]) {
00044     if (argc < 4) {
00045         cerr << "Usage: " << argv[0] << " <blocked_file> <index_file> <insert_file>" << endl;
00046         return 1;
00047     }
00048
00049     string blockedFile = argv[1];
00050     string indexFile = argv[2];
00051     string insertFile = argv[3];
00052
00053     // Load index from file
00054     map<string, int> index = loadIndex(indexFile);
00055     if (index.empty()) {
00056         cerr << "Empty or missing index file." << endl;
00057         return 1;
00058     }
00059
00060     // Open blocked sequence set file
00061     fstream data(blockedFile, ios::in | ios::out | ios::binary);
00062     if (!data.is_open()) {
00063         cerr << "Failed to open blocked file." << endl;
00064         return 1;
00065     }
00066
00067     // Read header
00068     HeaderBuffer3 hb;
00069     if (!hb.read(data)) {
00070         cerr << "Failed to read header." << endl;
00071         return 1;
00072     }
00073
00074     int blockSize = hb.header.blockSize;
00075     int nextRBN = hb.header.blockCount; // Start at next unused block
00076     int inserted = 0; // Count of records inserted
00077
00078     // Open file of new records to insert
00079     ifstream newRecords(insertFile);
00080     string newRec;
00081     while (getline(newRecords, newRec)) {
00082         string newKey = extractKey(newRec);
00083         auto it = index.lower_bound(newKey);

```

```

00097         int targetRBN = (it != index.end()) ? it->second : (--index.end())->second;
00098
00099         // Load the target block and insert new record
00100         Block blk;
00101         BlockBuffer::readBlock(static_cast<iostream&>(data), blk, targetRBN, blockSize);
00102         blk.records.push_back(newRec);
00103
00104         // Keep block sorted
00105         sort(blk.records.begin(), blk.records.end(), [](const string& a, const string& b) {
00106             return extractKey(a) < extractKey(b);
00107         });
00108
00109         // Estimate block size
00110         stringstream sizeSim;
00111         for (auto& r : blk.records) sizeSim << r << '\n';
00112
00113         if (static_cast<int>(sizeSim.str().size()) < blockSize) {
00114             // Block fits, update it
00115             blk.recordCount = blk.records.size();
00116             BlockBuffer::writeBlock(data, blk, targetRBN, blockSize);
00117         } else {
00118             // Split the block into two
00119             vector<string> left(blk.records.begin(), blk.records.begin() + blk.records.size() / 2);
00120             vector<string> right(blk.records.begin() + blk.records.size() / 2, blk.records.end());
00121
00122             Block leftBlock = { (int)left.size(), blk.prevRBN, nextRBN, left };
00123             Block rightBlock = { (int)right.size(), targetRBN, blk.nextRBN, right };
00124
00125             BlockBuffer::writeBlock(data, leftBlock, targetRBN, blockSize);
00126             BlockBuffer::writeBlock(data, rightBlock, nextRBN, blockSize);
00127
00128             // Update index with new keys
00129             string highLeft = extractKey(left.back());
00130             string highRight = extractKey(right.back());
00131             index.erase(highLeft);
00132             index[highLeft] = targetRBN;
00133             index[highRight] = nextRBN;
00134
00135             ++nextRBN;
00136             ++hb.header.blockCount;
00137             cout << "Split block " << targetRBN << " → " << nextRBN - 1 << endl;
00138         }
00139         ++hb.header.recordCount;
00140         ++inserted;
00141     }
00142
00143     // Rewind and update header in file
00144     data.seekp(0);
00145     std::ofstream outStream(blockedFile, ios::in | ios::out | ios::binary);
00146     hb.write(outStream);
00147
00148     // Save updated index
00149     writeIndex(indexFile, index);
00150
00151     cout << "Inserted " << inserted << " records." << endl;
00152     return 0;
00153 }

```

5.31 main.cpp File Reference

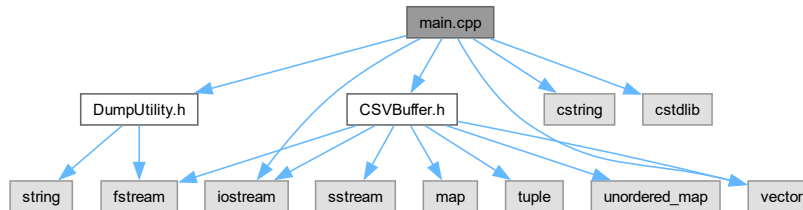
Main application to process Zip Code data from CSV or length-indicated file.

```

#include "CSVBuffer.h"
#include "DumpUtility.h"
#include <iostream>
#include <vector>
#include <cstring>
#include <cstdlib>

```


Include dependency graph for main.cpp:



Functions

- `int main (int argc, char *argv[])`

Main driver function.

5.31.1 Detailed Description

Main application to process Zip Code data from CSV or length-indicated file.

Main application to process Zip Code data using blocked sequence set structure.

Uses [CSVBuffer](#) to generate a state-wise table of extreme Zip Codes or convert CSV to length-indicated format.

Supports CSV conversion, summary generation, zip code search, record insertion, deletion, and dump features for blocked sequence set files.

Definition in file [main.cpp](#).

5.31.2 Function Documentation

5.31.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Main driver function.

Parameters

<i>argc</i>	Argument count.
<i>argv</i>	Argument vector.

Returns

Exit code.

< List of zip codes for search.

< Input data file name.

< Default index file name.

< File containing records to insert.

< File containing records to delete.

< Flag for physical dump.

< Flag for logical dump.

< Flag for insertion.

< Flag for deletion.

< Flag for zip code search.

Parse command-line arguments.

< Add zip code from argument.

< Assume non-flag argument is input file.

Validate that an input file was specified.

Perform zip code search if applicable.

< Load file into [CSVBuffer](#).

< Perform zip code search.

Dump blocks in physical order if flag set.

Dump blocks in logical order if flag set.

Run InsertRecord program if insertion requested.

< Execute external insert program.

Run DeleteRecord program if deletion requested.

< Execute external delete program.

If no arguments matched, enter interactive mode.

Option 1: Convert CSV to length-indicated format.

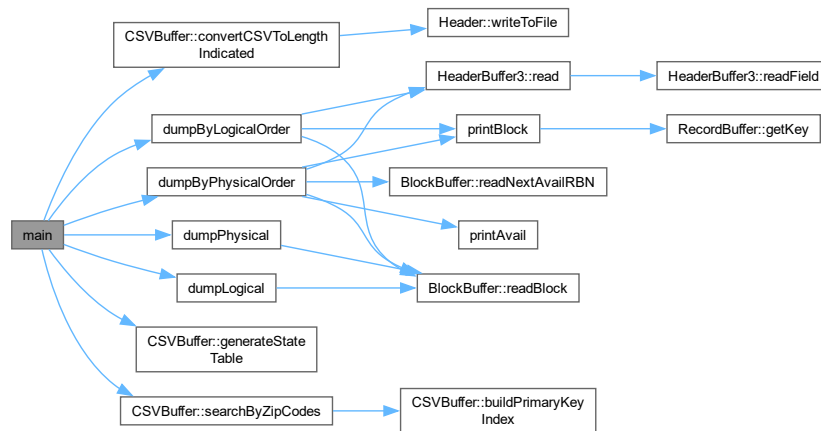
Option 2: Generate extreme zip summary per state.

< Generate summary report.

< Success.

Definition at line 21 of file [main.cpp](#).

Here is the call graph for this function:



5.32 main.cpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #include "CSVBuffer.h"
00007 #include "DumpUtility.h"
00008 #include <iostream>
00009 #include <vector>
00010 #include <cstring>
00011 #include <cstdlib>
00012
00013 using namespace std;
00014
00021 int main(int argc, char* argv[]) {
00022     vector<int> zipCodes;
00023     string inputFile;
00024     string indexFile = "zipcode.idx";
00025     string insertFile;
00026     string deleteFile;
00027     bool dumpPhysical = false;
00028     bool dumpLogical = false;
00029     bool runInsert = false;
00030     bool runDelete = false;
00031     bool runSearch = false;
00032
00036     for (int i = 1; i < argc; ++i) {
00037         string arg = argv[i];
00038         if (arg.rfind("-z", 0) == 0 || arg.rfind("-Z", 0) == 0) {
00039             try {
00040                 zipCodes.push_back(stoi(arg.substr(2)));
00041                 runSearch = true;
00042             } catch (...) {
00043                 cerr << "Invalid zip code argument: " << arg << endl;
00044                 return 1;
00045             }
00046         } else if (arg == "--dump-physical") {
00047             dumpPhysical = true;
00048         } else if (arg == "--dump-logical") {
00049             dumpLogical = true;
00050         } else if (arg == "--insert" && i + 1 < argc) {
00051             insertFile = argv[++i];
00052             runInsert = true;
00053         } else if (arg == "--delete" && i + 1 < argc) {
00054             deleteFile = argv[++i];
00055             runDelete = true;
00056         } else {

```

```

00057         inputFile = arg;
00058     }
00059 }
00060
00062 if (inputFile.empty()) {
00063     cerr << "Error: Must specify input file." << endl;
00064     return 1;
00065 }
00066
00068 if (runSearch && !zipCodes.empty()) {
00069     try {
00070         CSVBuffer buffer(inputFile);
00071         buffer.searchByZipCodes(zipCodes);
00072     } catch (const exception& e) {
00073         cerr << "Error: " << e.what() << endl;
00074         return 1;
00075     }
00076 }
00077
00079 if (dumpPhysical) {
00080     dumpByPhysicalOrder(inputFile);
00081 }
00082
00084 if (dumpLogical) {
00085     dumpByLogicalOrder(inputFile);
00086 }
00087
00089 if (runInsert && !insertFile.empty()) {
00090     string cmd = "./InsertRecord " + inputFile + " " + indexFile + " " + insertFile;
00091     system(cmd.c_str());
00092 }
00093
00095 if (runDelete && !deleteFile.empty()) {
00096     string cmd = "./DeleteRecord " + inputFile + " " + indexFile + " " + deleteFile;
00097     system(cmd.c_str());
00098 }
00099
00101 if (!runSearch && !runInsert && !runDelete && !dumpLogical && !dumpPhysical) {
00102     cout << "Choose operation:\n1. Convert CSV to length-indicated file\n2. Process file and
generate summary\nEnter choice (1 or 2): ";
00103     int choice;
00104     cin >> choice;
00105
00107     if (choice == 1) {
00108         string inputFile, outputFile;
00109         cout << "Enter input CSV filename: ";
00110         cin >> inputFile;
00111         cout << "Enter output filename for length-indicated file: ";
00112         cin >> outputFile;
00113         CSVBuffer::convertCSVTolengthIndicated(inputFile, outputFile);
00114         cout << "Conversion complete.\n";
00115     }
00117     else if (choice == 2) {
00118         string inputFile;
00119         cout << "Enter filename to process: ";
00120         cin >> inputFile;
00121         try {
00122             CSVBuffer buffer(inputFile);
00123             buffer.generateStateTable();
00124         } catch (const exception& e) {
00125             cerr << "Error: " << e.what() << endl;
00126             return 1;
00127         }
00128     } else {
00129         cout << "Invalid choice. Exiting.\n";
00130         return 1;
00131     }
00132 }
00133
00134 return 0;
00135 }

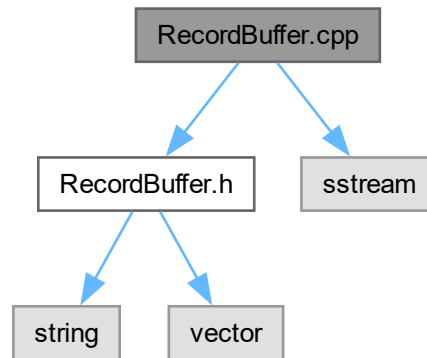
```

5.33 README.md File Reference

5.34 RecordBuffer.cpp File Reference

Implements the [RecordBuffer](#) class for parsing CSV records.

```
#include "RecordBuffer.h"
#include <sstream>
Include dependency graph for RecordBuffer.cpp:
```



5.34.1 Detailed Description

Implements the [RecordBuffer](#) class for parsing CSV records.

Definition in file [RecordBuffer.cpp](#).

5.35 RecordBuffer.cpp

[Go to the documentation of this file.](#)

```

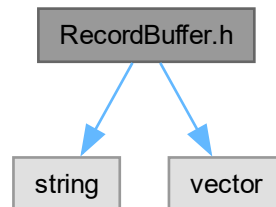
00001
00003
00004 #include "RecordBuffer.h"
00005 #include <sstream>
00006
00007 RecordBuffer::RecordBuffer(const std::string& record) {
00008     std::stringstream ss(record);
00009     std::string field;
00010     while (getline(ss, field, ',')) {
00011         fields.push_back(field);
00012     }
00013 }
00014
00015 std::string RecordBuffer::getKey() const {
00016     return fields.empty() ? "" : fields[0];
00017 }
00018
00019 std::vector<std::string> RecordBuffer::getFields() const {
00020     return fields;
00021 }
```

5.36 RecordBuffer.h File Reference

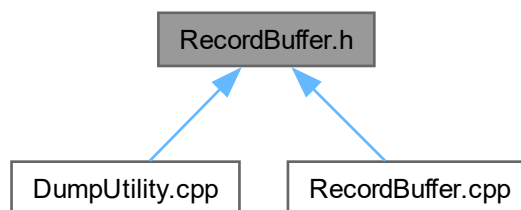
Provides a utility class to unpack a record string into fields.

```
#include <string>
#include <vector>
```

Include dependency graph for RecordBuffer.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [RecordBuffer](#)
[RecordBuffer](#) class extracts fields from a comma-separated record.

5.36.1 Detailed Description

Provides a utility class to unpack a record string into fields.

Definition in file [RecordBuffer.h](#).

5.37 RecordBuffer.h

[Go to the documentation of this file.](#)

```

00001
00003
00004 #ifndef RECORDBUFFER_H
00005 #define RECORDBUFFER_H
00006
00007 #include <string>
00008 #include <vector>
00009
00011 class RecordBuffer {
00012 public:
00013     RecordBuffer(const std::string& record);
00016
00019     std::string getKey() const;
00020
00023     std::vector<std::string> getFields() const;
00024
00025 private:
00026     std::vector<std::string> fields;
00027 };
00028
00029 #endif // RECORDBUFFER_H

```

5.38 SearchIndex.cpp File Reference

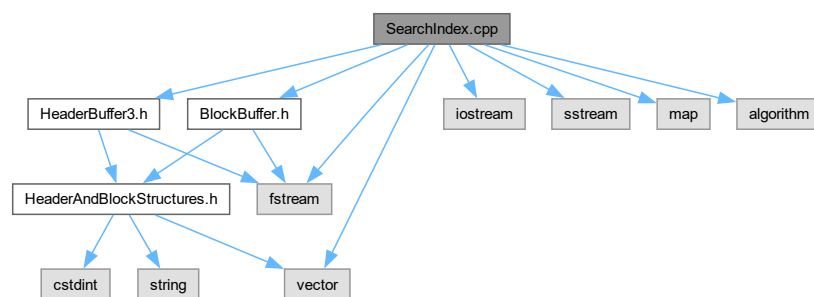
Searches for ZIP codes in a blocked sequence set using a primary key index file.

```

#include "HeaderBuffer3.h"
#include "BlockBuffer.h"
#include <fstream>
#include <iostream>
#include <sstream>
#include <vector>
#include <map>
#include <algorithm>

```

Include dependency graph for SearchIndex.cpp:



Functions

- string [extractKey](#) (const string &record)
Extracts the primary key (ZIP code) from a record.
- map< string, int > [loadIndex](#) (const string &indexFile)
Loads the index file into a sorted map.

- int [locateBlock](#) (const map< string, int > &index, const string &searchKey)
Finds the smallest RBN whose key is $\geq$ searchKey.
- int [main](#) (int argc, char *argv[])
Entry point to search ZIP codes using -z##### flags.

5.38.1 Detailed Description

Searches for ZIP codes in a blocked sequence set using a primary key index file.

Definition in file [SearchIndex.cpp](#).

5.38.2 Function Documentation

5.38.2.1 [extractKey\(\)](#)

```
string extractKey (
    const string & record)
```

Extracts the primary key (ZIP code) from a record.

Parameters

<i>record</i>	A comma-separated ZIP record string.
---------------	--------------------------------------

Returns

The first field (ZIP code) from the record.

Definition at line 20 of file [SearchIndex.cpp](#).

Here is the caller graph for this function:



5.38.2.2 [loadIndex\(\)](#)

```
map< string, int > loadIndex (
    const string & indexFile)
```

Loads the index file into a sorted map.

Parameters

<i>indexFile</i>	The filename of the index file to load.
------------------	---

Returns

A map from ZIP key to block RBN.

Definition at line 32 of file [SearchIndex.cpp](#).

Here is the caller graph for this function:



5.38.2.3 locateBlock()

```
int locateBlock (  
    const map< string, int > & index,  
    const string & searchKey)
```

Finds the smallest RBN whose key is $\geq$ searchKey.

Parameters

<i>index</i>	The in-memory map of key-to-RBN.
<i>searchKey</i>	The ZIP code key to look up.

Returns

The corresponding block RBN or -1 if not found.

Definition at line 57 of file [SearchIndex.cpp](#).

Here is the caller graph for this function:



5.38.2.4 main()

```
int main (
    int argc,
    char * argv[])
```

Entry point to search ZIP codes using -z##### flags.

Parameters

<i>argc</i>	Number of command-line arguments.
<i>argv</i>	Argument vector. Expects: <blocked_file> <index_file> -z##### [-z##### ...]

Returns

int Exit code.

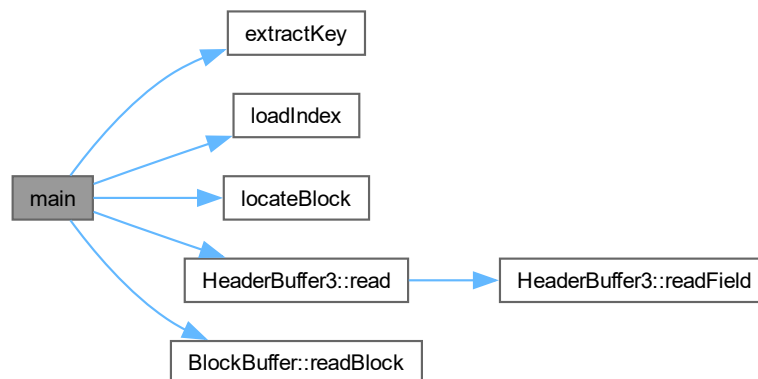
< File containing blocked ZIP records.

< File containing key-to-RBN index.

< ZIP codes passed via -z##### flags.

Definition at line 69 of file [SearchIndex.cpp](#).

Here is the call graph for this function:



5.39 SearchIndex.cpp

[Go to the documentation of this file.](#)

```

00001
00003
00004 #include "HeaderBuffer3.h"
00005 #include "BlockBuffer.h"
00006 #include <fstream>
00007 #include <iostream>
00008 #include <sstream>
00009 #include <vector>
00010 #include <map>
00011 #include <algorithm>
00012
00013 using namespace std;
00014
00020 string extractKey(const string& record) {
00021     stringstream ss(record);
00022     string key;
00023     getline(ss, key, ',');
00024     return key;
00025 }
00026
00032 map<string, int> loadIndex(const string& indexFile) {
00033     map<string, int> index;
00034     ifstream in(indexFile);
00035     if (!in.is_open()) {
00036         cerr << "Error opening index file." << endl;
00037         return index;
00038     }
00039     string line;
00040     while (getline(in, line)) {
00041         stringstream ss(line);
00042         string key;
00043         int rbn;
00044         getline(ss, key, ',');
00045         ss >> rbn;
00046         index[key] = rbn;
00047     }
00048     return index;
00049 }
00050
00057 int locateBlock(const map<string, int>& index, const string& searchKey) {
00058     auto it = index.lower_bound(searchKey);
00059     if (it == index.end()) return -1;
00060     return it->second;
00061 }
00062
00069 int main(int argc, char* argv[]) {
00070     if (argc < 3) {
00071         cerr << "Usage: " << argv[0] << " <blocked_file> <index_file> -z##### [-z##### ...]" << endl;
00072         return 1;
00073     }
00074
00075     string blockedFile = argv[1];
00076     string indexFile = argv[2];
00077     vector<string> searchZips;
00078
00079     // Parse ZIP code flags from command line.
00080     for (int i = 3; i < argc; ++i) {
00081         if (strncmp(argv[i], "-z", 2) == 0 || strncmp(argv[i], "-Z", 2) == 0) {
00082             searchZips.push_back(argv[i] + 2);
00083         }
00084     }
00085
00086     auto index = loadIndex(indexFile);
00087     if (index.empty()) return 1;
00088
00089     ifstream data(blockedFile, ios::binary);
00090     if (!data.is_open()) {
00091         cerr << "Could not open blocked file." << endl;
00092         return 1;
00093     }
00094
00095     HeaderBuffer3 hb;
00096     if (!hb.read(data)) {
00097         cerr << "Failed to read header from blocked file." << endl;
00098         return 1;
00099     }
00100     int blockSize = hb.header.blockSize;
00101
00102     // Search for each ZIP in the provided flags
00103     for (const string& zip : searchZips) {
00104         int rbn = locateBlock(index, zip);
00105         if (rbn == -1) {

```

```
00106         cout << "Zip Code " << zip << " not found in index." << endl;
00107         continue;
00108     }
00109
00110     Block blk;
00111     if (!BlockBuffer::readBlock(data, blk, rbn, blockSize)) {
00112         cerr << "Failed to read block RBN " << rbn << endl;
00113         continue;
00114     }
00115
00116     bool found = false;
00117     for (const auto& record : blk.records) {
00118         if (extractKey(record) == zip) {
00119             cout << "Found: " << record << endl;
00120             found = true;
00121             break;
00122         }
00123     }
00124     if (!found) {
00125         cout << "Zip Code " << zip << " not found in data block." << endl;
00126     }
00127 }
00128
00129 return 0;
00130 }
```

Index

- AvailBlock, [9](#)
 - AvailBlock, [9](#)
 - blankFill, [10](#)
 - nextAvailRBN, [10](#)
 - recordCount, [10](#)
- availListHeadRBN
 - Header, [25](#)
- blankFill
 - AvailBlock, [10](#)
- Block, [10](#)
 - nextRBN, [11](#)
 - prevRBN, [11](#)
 - recordCount, [11](#)
 - records, [11](#)
- BlockBuffer, [11](#)
 - readBlock, [12](#)
 - readNextAvailRBN, [13](#)
 - writeAvailBlock, [13](#)
 - writeBlock, [14](#)
- BlockBuffer.cpp, [37](#)
- BlockBuffer.h, [39](#)
- blockCount
 - Header, [25](#)
- blockSize
 - Header, [25](#)
- BuildIndex.cpp, [40](#)
 - extractKey, [41](#)
 - main, [41](#)
- buildPrimaryKeyIndex
 - CSVBuffer, [17](#)
- convertCSVToLengthIndicated
 - CSVBuffer, [17](#)
- ConvertToBlocked.cpp, [43](#)
 - main, [44](#)
- county
 - ZipRecord, [35](#)
- CSVBuffer, [15](#)
 - buildPrimaryKeyIndex, [17](#)
 - convertCSVToLengthIndicated, [17](#)
 - CSVBuffer, [16](#)
 - file, [20](#)
 - generateStateTable, [18](#)
 - getRecords, [18](#)
 - loadLengthIndicatedRecords, [18](#)
 - loadRecords, [19](#)
 - records, [20](#)
 - searchByZipCodes, [19](#)
- CSVBuffer.cpp, [46](#)
- CSVBuffer.h, [50](#)
- DeleteRecord.cpp, [52](#)
 - extractKey, [53](#)
 - loadIndex, [53](#)
 - main, [53](#)
 - writeIndex, [54](#)
- DumpBlocked.cpp, [56](#)
 - dumpLogical, [57](#)
 - dumpPhysical, [57](#)
 - main, [58](#)
- dumpByLogicalOrder
 - DumpUtility.cpp, [61](#)
 - DumpUtility.h, [66](#)
- dumpByPhysicalOrder
 - DumpUtility.cpp, [61](#)
 - DumpUtility.h, [66](#)
- dumpLogical
 - DumpBlocked.cpp, [57](#)
- dumpPhysical
 - DumpBlocked.cpp, [57](#)
- DumpUtility.cpp, [60](#)
 - dumpByLogicalOrder, [61](#)
 - dumpByPhysicalOrder, [61](#)
 - printAvail, [62](#)
 - printBlock, [63](#)
- DumpUtility.h, [65](#)
 - dumpByLogicalOrder, [66](#)
 - dumpByPhysicalOrder, [66](#)
- extractKey
 - BuildIndex.cpp, [41](#)
 - DeleteRecord.cpp, [53](#)
 - InsertRecord.cpp, [76](#)
 - SearchIndex.cpp, [88](#)
- Field, [21](#)
 - format, [21](#)
 - isPrimaryKey, [21](#)
 - name, [21](#)
 - type, [22](#)
- fields
 - Header, [25](#)
 - RecordBuffer, [35](#)
- fieldsPerRecord
 - Header, [26](#)
- file
 - CSVBuffer, [20](#)
- fileStructureType
 - Header, [26](#)

- format
 - Field, 21
- generateStateTable
 - CSVBuffer, 18
- getFields
 - RecordBuffer, 34
- getKey
 - RecordBuffer, 34
- getRecords
 - CSVBuffer, 18
- Header, 22
 - availListHeadRBN, 25
 - blockCount, 25
 - blockSize, 25
 - fields, 25
 - fieldsPerRecord, 26
 - fileStructureType, 26
 - Header, 24
 - headerSize, 26
 - indexFileName, 26
 - indexSchema, 26
 - minBlockCapacity, 26
 - primaryKey, 27
 - primaryKeyFileName, 27
 - primaryKeyOrdinal, 27
 - readFromFile, 24
 - recordCount, 27
 - recordSizeFieldLength, 27
 - sequenceListHeadRBN, 27
 - sizeFormatType, 28
 - sizeInBytes, 28
 - stale, 28
 - version, 28
 - writeToFile, 24
- header
 - HeaderBuffer3, 33
- HeaderAndBlockStructures.h, 67
- HeaderBuffer.h, 68
- HeaderBuffer3, 29
 - header, 33
 - read, 29
 - readField, 30
 - write, 31
 - writeField, 32
- HeaderBuffer3.cpp, 71
- HeaderBuffer3.h, 73
- headerSize
 - Header, 26
- indexFileName
 - Header, 26
- indexSchema
 - Header, 26
- InsertRecord.cpp, 75
 - extractKey, 76
 - loadIndex, 76
 - main, 77
 - writeIndex, 78
- isPrimaryKey
 - Field, 21
- latitude
 - ZipRecord, 35
- loadIndex
 - DeleteRecord.cpp, 53
 - InsertRecord.cpp, 76
 - SearchIndex.cpp, 88
- loadLengthIndicatedRecords
 - CSVBuffer, 18
- loadRecords
 - CSVBuffer, 19
- locateBlock
 - SearchIndex.cpp, 89
- longitude
 - ZipRecord, 36
- main
 - BuildIndex.cpp, 41
 - ConvertToBlocked.cpp, 44
 - DeleteRecord.cpp, 53
 - DumpBlocked.cpp, 58
 - InsertRecord.cpp, 77
 - main.cpp, 81
 - SearchIndex.cpp, 89
- main.cpp, 80
 - main, 81
- minBlockCapacity
 - Header, 26
- name
 - Field, 21
- nextAvailRBN
 - AvailBlock, 10
- nextRBN
 - Block, 11
- placeName
 - ZipRecord, 36
- prevRBN
 - Block, 11
- primaryKey
 - Header, 27
- primaryKeyFileName
 - Header, 27
- primaryKeyOrdinal
 - Header, 27
- printAvail
 - DumpUtility.cpp, 62
- printBlock
 - DumpUtility.cpp, 63
- read
 - HeaderBuffer3, 29
- readBlock
 - BlockBuffer, 12
- readField

- HeaderBuffer3, 30
- readFromFile
 - Header, 24
- README.md, 84
- readNextAvailRBN
 - BlockBuffer, 13
- RecordBuffer, 33
 - fields, 35
 - getFields, 34
 - getKey, 34
 - RecordBuffer, 33
- RecordBuffer.cpp, 84
- RecordBuffer.h, 85
- recordCount
 - AvailBlock, 10
 - Block, 11
 - Header, 27
- records
 - Block, 11
 - CSVBuffer, 20
- recordSizeFieldLength
 - Header, 27
- searchByZipCodes
 - CSVBuffer, 19
- SearchIndex.cpp, 87
 - extractKey, 88
 - loadIndex, 88
 - locateBlock, 89
 - main, 89
- sequenceListHeadRBN
 - Header, 27
- sizeFormatType
 - Header, 28
- sizeInBytes
 - Header, 28
- stale
 - Header, 28
- state
 - ZipRecord, 36
- type
 - Field, 22
- version
 - Header, 28
- write
 - HeaderBuffer3, 31
- writeAvailBlock
 - BlockBuffer, 13
- writeBlock
 - BlockBuffer, 14
- writeField
 - HeaderBuffer3, 32
- writeIndex
 - DeleteRecord.cpp, 54
 - InsertRecord.cpp, 78
- writeToFile
 - Header, 24
- zipCode
 - ZipRecord, 36
- ZipCode3.0, 1
- ZipRecord, 35
 - county, 35
 - latitude, 35
 - longitude, 36
 - placeName, 36
 - state, 36
 - zipCode, 36